CHƯƠNG 8
XÂY DỰNG ỨNG DỤNG AI TẠO SINH

Sau khi học xong chương này, người học có thể:

- *Mô tả được quy trình xây dựng ứng dụng AI tạo sinh từ ý tưởng đến sản phẩm, bao gồm năm giai đoạn chính và các yêu cầu chức năng, phi chức năng đặc thù của dự án GenAI.*
- *Phân biệt và lựa chọn được ba mô hình tích hợp AI (Direct API Call, Function Calling, Agent-based) phù hợp với từng bài toán cụ thể, và thiết kế được API cho ứng dụng GenAI.*
- *Xây dựng được chiến lược kiểm thử toàn diện cho ứng dụng GenAI, bao gồm kiểm thử phần mềm truyền thống và kiểm thử đặc thù AI, với các chỉ số đánh giá phù hợp và cơ chế phát hiện, giảm thiểu hallucination.*
- *Phát triển được một dự án ứng dụng AI tạo sinh hoàn chỉnh theo quy trình chín bước, từ phát biểu bài toán, thiết kế kiến trúc, xây dựng prototype, đến triển khai production và cải thiện liên tục.*
- *Tích hợp được kiến thức từ các chương trước (prompt engineering, kiến trúc hệ thống, thiết kế dữ liệu, RAG) vào một dự án GenAI thực tế, có khả năng vận hành trong môi trường production.*

8.1 Quy trình xây dựng ứng dụng AI tạo sinh

Sau khi hoàn thành mục này, sinh viên có thể:

- *Mô tả được quy trình xây dựng ứng dụng AI tạo sinh từ ý tưởng đến sản phẩm, bao gồm năm giai đoạn chính.*
- *Phân biệt được yêu cầu chức năng và yêu cầu phi chức năng trong dự án GenAI, và giải thích được tầm quan trọng của từng loại.*
- *Phác thảo được kiến trúc tổng thể của một ứng dụng GenAI theo mô hình client-server với các thành phần chính.*
- *Giải thích được vì sao quy trình phát triển ứng dụng GenAI cần theo hướng lặp (iterative) thay vì tuyến tính (linear).*
- *Nhận diện được những thách thức chính khi đưa ứng dụng GenAI từ prototype lên production.*

8.1.1 Khởi động: Từ ý tưởng đến sản phẩm – hành trình khác biệt

Hãy hình dung bạn vừa nảy ra một ý tưởng: xây dựng một trợ lý học tập thông minh có thể giải thích các khái niệm lập trình, đưa ra bài tập phù hợp với trình độ người học, và chấm điểm câu trả lời tự luận. Với quy trình phát triển phần mềm truyền thống, bạn sẽ bắt đầu bằng việc phân tích yêu cầu chi tiết, thiết kế cơ sở dữ liệu, viết các quy tắc nghiệp vụ, và kiểm thử từng nhánh logic. Nhưng với một ứng dụng AI tạo sinh – nơi

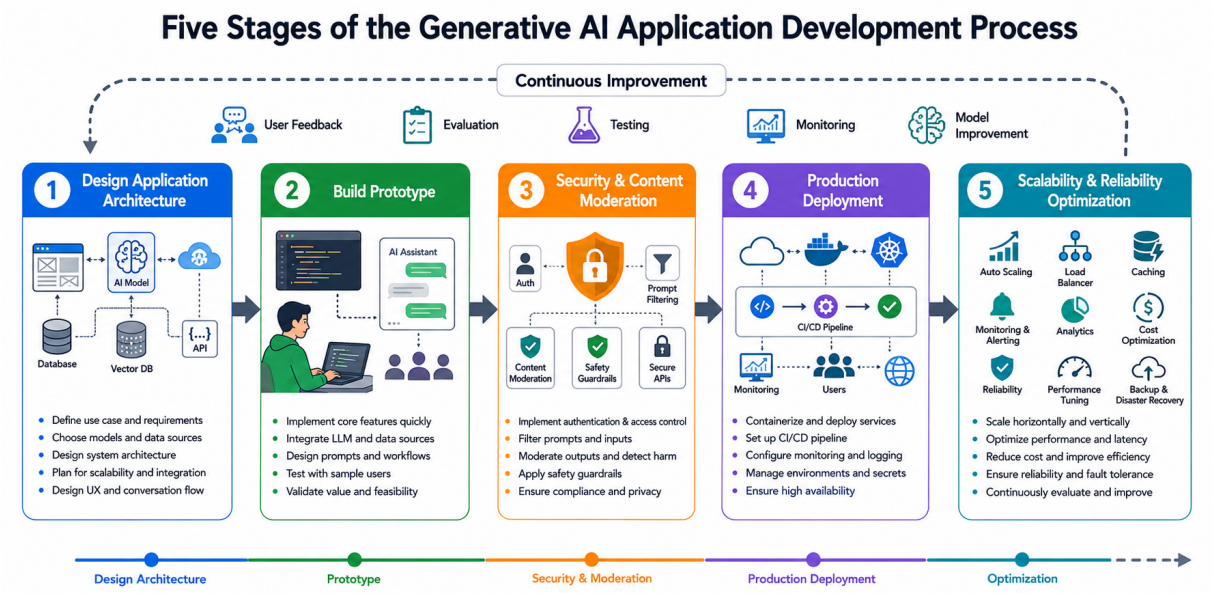
phần lớn "trí thông minh" đến từ mô hình ngôn ngữ lớn (LLM) thay vì từ code – hành trình đó có còn giống như trước?

Câu trả lời là: vừa giống, vừa khác. Bạn vẫn cần phân tích yêu cầu, thiết kế hệ thống, viết code, và kiểm thử. Nhưng cách bạn tiếp cận từng bước, thứ tự ưu tiên, và tiêu chí thành công sẽ thay đổi đáng kể. Một khảo sát thực tế cho thấy nhiều ứng dụng GenAI không vượt qua được giai đoạn prototype – không phải vì ý tưởng kém, mà vì đội ngũ phát triển đánh giá thấp những thách thức về khả năng mở rộng, chi phí vận hành và tính không xác định của đầu ra mô hình.

Mục này trình bày quy trình xây dựng ứng dụng AI tạo sinh một cách có hệ thống, từ lúc định hình ý tưởng cho đến khi triển khai sản phẩm thực tế. Chúng ta sẽ sử dụng một case study xuyên suốt – ứng dụng SkillGenie, một trợ lý học tập thông minh được hỗ trợ bởi GenAI – để minh họa cho từng giai đoạn của quy trình.

8.1.2 Năm giai đoạn của quy trình xây dựng ứng dụng GenAI

Quy trình xây dựng ứng dụng AI tạo sinh có thể được tổ chức thành năm giai đoạn chính, như minh họa trong Hình 8.1. Đây không phải là một quy trình tuyến tính cứng nhắc – trong thực tế, bạn sẽ thường xuyên quay lại các giai đoạn trước dựa trên kết quả kiểm thử và phản hồi từ người dùng.



Hình 8.1: Năm giai đoạn của quy trình xây dựng ứng dụng AI tạo sinh: Thiết kế kiến trúc ứng dụng → Xây dựng prototype → Triển khai bảo mật và kiểm duyệt nội dung → Triển khai production → Tối ưu hóa cho khả năng mở rộng và độ tin cậy. Mũi tên nét đứt thể hiện tính lặp (iterative) của quy trình.

a. Giai đoạn 1: Thiết kế kiến trúc ứng dụng

Giai đoạn đầu tiên tập trung vào việc trả lời ba câu hỏi nền tảng: *Ứng dụng sẽ làm gì?*, *Nó sẽ được xây dựng như thế nào?*, và *Ai sẽ sử dụng nó?*. Đây là lúc bạn chuyển từ một ý tưởng mơ hồ sang một bản thiết kế khả thi.

Trước hết, bạn cần xác định rõ các yêu cầu chức năng (functional requirements) – những gì ứng dụng phải làm được từ góc nhìn của người dùng. Với SkillGenie, các yêu cầu chức năng bao gồm: hỗ trợ đầu vào đa phương thức (văn bản và âm thanh), cho phép

người dùng chọn chủ đề học tập từ danh sách định sẵn, cung cấp hai chế độ học (tự học và có hướng dẫn), và đánh giá câu trả lời kèm phản hồi chi tiết. Mỗi yêu cầu chức năng cần được diễn đạt ở mức đủ cụ thể để có thể kiểm chứng được – "người dùng có thể yêu cầu giải thích một khái niệm và nhận được câu trả lời trong vòng 5 giây" tốt hơn nhiều so với "ứng dụng trả lời câu hỏi của người dùng".

Tiếp theo, bạn cần xác định các yêu cầu phi chức năng (non-functional requirements) – những ràng buộc về chất lượng và vận hành. Với ứng dụng GenAI, nhóm yêu cầu này đặc biệt quan trọng vì chúng ảnh hưởng trực tiếp đến trải nghiệm người dùng và chi phí vận hành. Bảng 8.1 tóm tắt các yêu cầu phi chức năng điển hình cho một ứng dụng GenAI.

Loại yêu cầu	**Mô tả**	**Ví dụ với SkillGenie**
Xác thực & Phân quyền	Cơ chế đăng nhập, phân quyền theo vai trò	Người dùng đăng nhập qua email; giáo viên có quyền xem báo cáo lớp học
Bảo mật dữ liệu	Mã hóa, kiểm soát truy cập, tuân thủ quy định	Mã hóa dữ liệu hội thoại; không gửi dữ liệu cá nhân cho LLM bên ngoài
Hiệu năng	Thời gian phản hồi, throughput	Phản hồi câu hỏi trong $\leq$ 5 giây; hỗ trợ $\geq$ 100 người dùng đồng thời
Khả năng mở rộng	Tự động scale theo tải	Hệ thống tự động tăng/giảm GPU instances theo lưu lượng truy vấn
Ghi log & Giám sát	Theo dõi hoạt động hệ thống và mô hình	Ghi lại mọi lời gọi LLM API; cảnh báo nếu tỉ lệ lỗi $>$ 1%
Thông báo	Cơ chế gửi thông báo đến người dùng	Gửi email nhắc nhở lịch học; thông báo khi có phản hồi mới

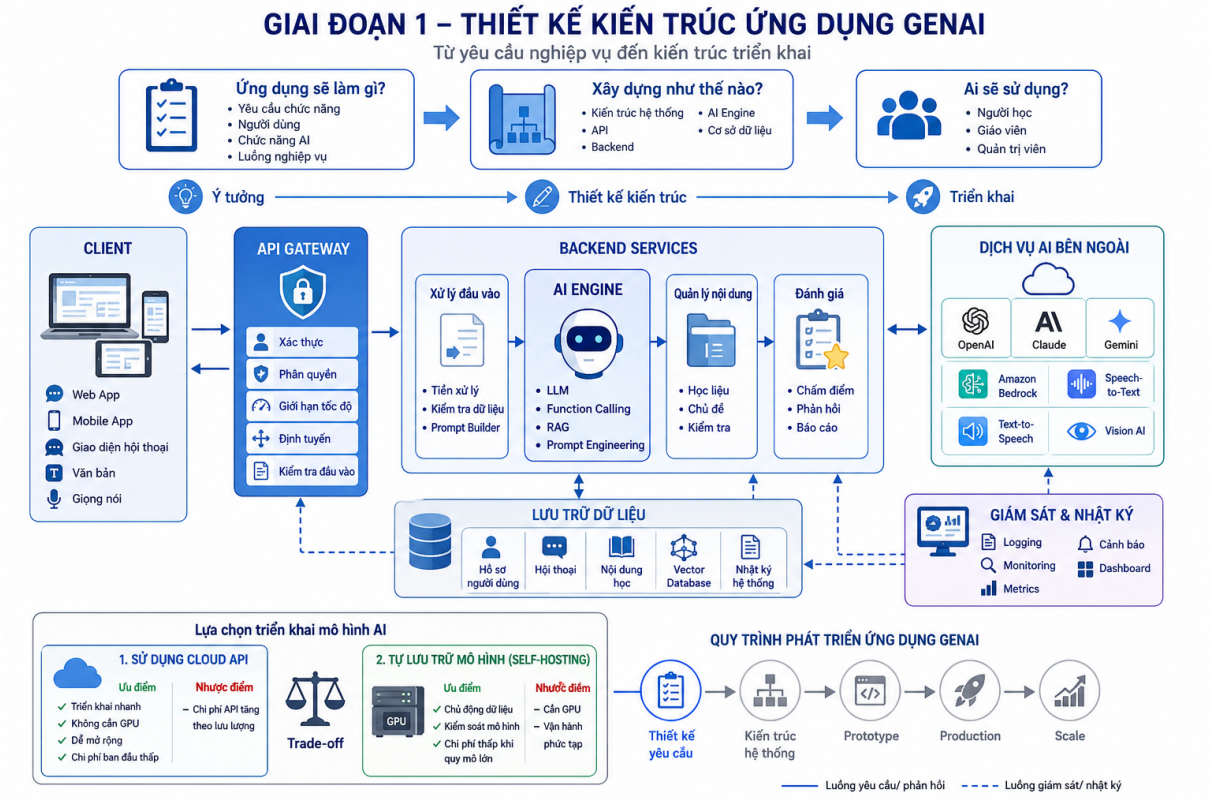
Bảng 8.1: Các yêu cầu phi chức năng điển hình cho ứng dụng GenAI

Sau khi xác định yêu cầu, bước tiếp theo là thiết kế kiến trúc tổng thể. Hình 8.2 minh họa kiến trúc client-server điển hình cho một ứng dụng GenAI. Kiến trúc này bao gồm sáu khối chính: (1) **Client** – giao diện người dùng (mobile app, web app); (2) **API Gateway** – điểm vào trung tâm, chịu trách nhiệm định tuyến, giới hạn tốc độ và xác thực yêu cầu; (3) **Backend Services** – nơi chứa logic nghiệp vụ, bao gồm các module xử lý đầu vào, AI Engine (tích hợp LLM), quản lý nội dung và đánh giá; (4) **Data Storage** – lưu trữ dữ liệu người dùng, nội dung học tập, và kho giải pháp; (5) **External Services** – các dịch vụ AI bên ngoài như LLM API, dịch vụ xử lý đa phương thức; (6) **Monitoring & Logging** – hạ tầng giám sát xuyên suốt tất cả các tầng.

Một quyết định kiến trúc quan trọng ở giai đoạn này là lựa chọn giữa việc tự lưu trữ mô hình (self-hosting) hay sử dụng dịch vụ đám mây (cloud-based inference). Với hầu hết dự án khởi đầu, sử dụng cloud API (như OpenAI, Anthropic, hay Amazon Bedrock) là lựa chọn hợp lý vì giảm đáng kể chi phí hạ tầng ban đầu và độ phức tạp vận hành. Tuy nhiên, khi ứng dụng mở rộng về quy mô, chi phí API có thể vượt chi phí tự lưu trữ – đây là một trade-off mà bạn cần liên tục đánh giá.

b. Giai đoạn 2: Xây dựng prototype hoạt động được

Mục tiêu của giai đoạn này không phải là xây dựng một sản phẩm hoàn chỉnh, mà là tạo ra một phiên bản chạy được với chức năng tối thiểu (MVP – Minimum Viable Product) để kiểm chứng ý tưởng. Một nguyên tắc quan trọng: **prototype càng sớm có**



Hình 8.2: Kiến trúc tổng thể của ứng dụng GenAI theo mô hình client-server. API Gateway đóng vai trò trung tâm điều phối, Backend Services chứa AI Engine để tích hợp LLM, và External Services cung cấp năng lực AI thông qua API.

người dùng thật trải nghiệm, bạn càng sớm biết được điều gì thực sự có giá trị và điều gì chỉ là ảo tưởng.

Trong quá trình xây dựng prototype, bạn sẽ cần: tích hợp mô hình AI (kết nối đến LLM API và xử lý phản hồi), xây dựng luồng tương tác người dùng cơ bản, và triển khai các chức năng xử lý đầu vào. Với SkillGenie, prototype ban đầu có thể chỉ hỗ trợ một chủ đề học tập, một chế độ học (tự học), và đầu vào văn bản thuần túy. Điều này nghe có vẻ hạn chế, nhưng nó đủ để bạn kiểm tra giả thuyết quan trọng nhất: liệu người dùng có thấy giá trị trong việc được một AI giải thích khái niệm lập trình hay không?

Khi xây dựng prototype, hãy giữ cho kiến trúc đủ đơn giản để có thể thay đổi nhanh chóng. Đừng đầu tư quá nhiều vào tối ưu hóa hay xử lý edge case ở giai đoạn này – những thứ đó sẽ đến sau khi bạn đã xác nhận được hướng đi đúng. Một sai lầm phổ biến là dành hàng tuần để hoàn thiện một tính năng trước khi cho người dùng dùng thử, chỉ để nhận ra rằng người dùng cần một thứ hoàn toàn khác.

c. Giai đoạn 3: Triển khai bảo mật và kiểm duyệt nội dung

Khi prototype đã được kiểm chứng và bạn quyết định tiến tới production, bảo mật và kiểm duyệt nội dung trở thành ưu tiên hàng đầu. Ứng dụng GenAI đối mặt với những thách thức bảo mật đặc thù mà ứng dụng truyền thống không có: prompt injection (kẻ tấn công chèn chỉ thị độc hại vào input người dùng để điều khiển mô hình), data leakage (mô hình vô tình tiết lộ dữ liệu nhạy cảm từ ngữ cảnh), và harmful content generation (mô hình sinh ra nội dung độc hại, phân biệt hoặc không phù hợp).

Các biện pháp bảo vệ chính bao gồm:

- **Xác thực đầu vào (input validation)**: sử dụng các moderation API như OpenAI Moderation API để phát hiện và chặn nội dung độc hại trước khi nó đến được mô hình. Đây là tuyến phòng thủ đầu tiên và quan trọng nhất.
- **System prompt cứng**: thiết kế system prompt bao gồm các quy tắc rõ ràng về những gì mô hình được phép và không được phép làm. Ví dụ: "Bạn là trợ lý học tập. Không trả lời các câu hỏi về bạo lực, chính trị, hoặc nội dung người lớn. Nếu được hỏi, hãy từ chối lịch sự."
- **Lọc đầu ra (output filtering)**: kiểm tra phản hồi của mô hình trước khi hiển thị cho người dùng. Nếu phản hồi chứa nội dung không phù hợp, hệ thống có thể chặn hoặc thay thế bằng một thông báo an toàn.
- **Kiểm soát truy cập**: đảm bảo mỗi người dùng chỉ có thể truy cập dữ liệu của chính họ, và các API key được quản lý an toàn (không bao giờ nhúng trực tiếp vào mã nguồn client-side).

Một nguyên tắc thiết kế quan trọng là **defense in depth** – không bao giờ dựa vào một cơ chế bảo vệ duy nhất. Nếu moderation API bỏ sót một input độc hại, system prompt và output filter vẫn có thể ngăn chặn thiệt hại.

d. Giai đoạn 4: Triển khai production

Triển khai production cho ứng dụng GenAI đòi hỏi sự kết hợp giữa các thực hành DevOps truyền thống và các yêu cầu đặc thù của hệ thống AI. Các thành phần chính của quy trình triển khai bao gồm:

Container hóa với Docker: Đóng gói toàn bộ ứng dụng – bao gồm backend services, dependencies, và cấu hình môi trường – vào các Docker container. Container hóa

đảm bảo ứng dụng chạy nhất quán trên mọi môi trường từ máy phát triển đến production, loại bỏ vấn đề "trên máy tôi nó chạy được mà" (works on my machine).

CI/CD Pipeline: Thiết lập quy trình tích hợp và triển khai liên tục để tự động hóa việc kiểm thử, build, và deploy mỗi khi có thay đổi mã nguồn. Một CI/CD pipeline điển hình cho GenAI bao gồm: chạy unit tests → kiểm tra chất lượng code → build Docker image → chạy integration tests (bao gồm cả gọi LLM API với test prompts) → deploy lên staging → smoke tests → deploy lên production. Khác với ứng dụng truyền thống, pipeline cho GenAI cần bao gồm các bước kiểm tra đặc thù như đánh giá chất lượng đầu ra mô hình và phát hiện regression về độ chính xác.

Triển khai đa vùng: Để đảm bảo độ trễ thấp và tính sẵn sàng cao, ứng dụng nên được triển khai tại nhiều vùng địa lý (multi-region deployment). Điều này đặc biệt quan trọng với ứng dụng GenAI vì thời gian phản hồi của LLM API có thể dao động đáng kể giữa các vùng.

e. Giai đoạn 5: Tối ưu hóa cho khả năng mở rộng và độ tin cậy

Sau khi triển khai, công việc chưa kết thúc. Ứng dụng GenAI trong môi trường production cần được liên tục giám sát và tối ưu. Ba trụ cột của giai đoạn này là:

1. **Giám sát hiệu năng mô hình**: theo dõi các chỉ số như thời gian phản hồi trung bình, tỉ lệ lỗi API, và phân phối độ dài phản hồi. Sự suy giảm đột ngột của một chỉ số có thể báo hiệu vấn đề với model provider hoặc thay đổi trong hành vi người dùng.
2. **Thu thập phản hồi người dùng**: cung cấp cơ chế để người dùng đánh giá chất lượng câu trả lời (thumbs up/down, xếp hạng sao). Dữ liệu phản hồi này là nguồn thông tin quý giá để cải thiện prompt, điều chỉnh tham số mô hình, và xác định các tình huống mà mô hình xử lý kém.
3. **Cải thiện liên tục**: dựa trên dữ liệu giám sát và phản hồi, liên tục điều chỉnh system prompt, cập nhật cơ sở tri thức, tinh chỉnh các tham số như `temperature` và `max\_tokens`, và thử nghiệm với các phiên bản mô hình mới hơn.

8.1.3 Quy trình lặp: Build – Measure – Learn

Một trong những khác biệt quan trọng nhất giữa phát triển ứng dụng truyền thống và ứng dụng GenAI là mức độ không chắc chắn. Trong ứng dụng truyền thống, nếu cùng một input luôn cho cùng một output, bạn có thể tự tin rằng hệ thống hoạt động đúng. Trong ứng dụng GenAI, cùng một prompt có thể tạo ra các phản hồi khác nhau – và không phải lúc nào bạn cũng biết trước phản hồi nào là "đúng".

Chính vì vậy, quy trình phát triển ứng dụng GenAI cần theo mô hình **Build – Measure – Learn** thay vì mô hình thác nước (waterfall) truyền thống. Cụ thể:

- **Build**: xây dựng một phiên bản với các giả thuyết rõ ràng về điều sẽ tạo ra giá trị cho người dùng.
- **Measure**: đo lường hành vi thực tế của hệ thống – không chỉ các chỉ số kỹ thuật (độ trễ, tỉ lệ lỗi) mà còn các chỉ số về chất lượng (độ chính xác, mức độ hài lòng của người dùng).
- **Learn**: rút ra bài học từ dữ liệu đo lường, điều chỉnh giả thuyết, và quyết định bước đi tiếp theo – cải thiện prompt, thay đổi mô hình, hay thậm chí xoay trục (pivot) sang hướng khác.

Vòng lặp này nên được thực hiện nhanh nhất có thể, đặc biệt trong giai đoạn đầu của dự án. Một tuần với ba vòng Build-Measure-Learn nhỏ có giá trị hơn nhiều so với một tháng xây dựng mà không có phản hồi.

8.1.4 Case study: Hành trình xây dựng SkillGenie

Để cụ thể hóa quy trình năm giai đoạn, hãy cùng xem xét cách đội ngũ phát triển SkillGenie – một ứng dụng trợ lý học tập thông minh – đã đi từ ý tưởng đến sản phẩm.

Bước 1: Phát biểu bài toán: Đội ngũ bắt đầu bằng việc xác định rõ vấn đề cần giải quyết: "Người học lập trình thường gặp khó khăn khi tự học vì không có ai giải thích khái niệm khó, không biết mình đang hiểu sai ở đâu, và thiếu lộ trình học tập cá nhân hóa." Từ đó, họ định nghĩa đề xuất giá trị cốt lõi: một trợ lý AI có thể giải thích khái niệm, đưa ra bài tập phù hợp và cung cấp phản hồi tức thì – mọi lúc, mọi nơi.

Bước 2: Thiết kế trải nghiệm người dùng: Đội ngũ phác thảo hành trình người dùng (user journey) gồm các bước: (a) chào mừng và chọn chủ đề; (b) chọn chế độ học (tự do hoặc có hướng dẫn); (c) tương tác với AI – đặt câu hỏi, nhận giải thích; (d) làm bài tập và nhận đánh giá; (e) xem tiến độ học tập.

Bước 3: Thiết kế hệ thống: Kiến trúc được thiết kế theo mô hình client-server với API Gateway làm trung tâm, bốn backend services chính (Input Processing, AI Engine, Lesson Management, Evaluation), và ba tầng lưu trữ dữ liệu (User Data, Content Database, Solution Repository).

Bước 4: Xây dựng prototype: Phiên bản đầu tiên chỉ hỗ trợ một chủ đề (Python cơ bản), chế độ tự học với văn bản, và tích hợp OpenAI API cho phần sinh nội dung. Prototype được đưa cho 10 người dùng thử nghiệm trong một tuần.

Bước 5: Học từ phản hồi: Kết quả thử nghiệm cho thấy người dùng đánh giá cao chất lượng giải thích, nhưng gặp khó khăn khi không biết nên hỏi gì tiếp theo. Đội ngũ đã thêm tính năng "câu hỏi gợi ý" – những câu hỏi tiếp theo được AI đề xuất dựa trên ngữ cảnh hội thoại – và tỉ lệ tương tác tăng 40%.

Bước 6: Tiến tới production: Sau ba vòng lặp Build-Measure-Learn, đội ngũ đã triển khai đầy đủ các lớp bảo mật (moderation API, system prompt cứng, output filter), đóng gói ứng dụng vào Docker, thiết lập CI/CD pipeline, và triển khai lên môi trường production với giám sát liên tục.

8.1.5 Từ quy trình đến tích hợp

Nắm vững quy trình xây dựng là bước đầu tiên. Câu hỏi tiếp theo là: làm thế nào để tích hợp mô hình AI – trái tim của ứng dụng GenAI – vào kiến trúc phần mềm một cách hiệu quả? Đây là chủ đề của mục tiếp theo, nơi chúng ta sẽ đi sâu vào các integration pattern, thiết kế API, và cơ chế function calling – những "mảnh ghép" kỹ thuật giúp biến kiến trúc trên giấy thành một hệ thống hoạt động được.

Trước khi chuyển sang mục tiếp theo, hãy tự kiểm tra: Bạn có thể liệt kê năm giai đoạn của quy trình xây dựng ứng dụng GenAI và giải thích điều gì làm cho giai đoạn 3 (bảo mật và kiểm duyệt nội dung) trở nên đặc biệt quan trọng đối với ứng dụng GenAI so với ứng dụng web truyền thống?

8.2 Tích hợp mô hình AI vào ứng dụng phần mềm

Sau khi hoàn thành mục này, sinh viên có thể:

- *Phân biệt được ba mô hình tích hợp AI chính: gọi API trực tiếp, function calling, và tích hợp dựa trên agent.*
- *Giải thích được vai trò của API Gateway trong kiến trúc ứng dụng GenAI và các chức năng chính của nó.*
- *Thiết kế được một API endpoint cho ứng dụng GenAI, bao gồm cấu trúc request/response và xử lý lỗi.*
- *Mô tả được cơ chế function calling và phân biệt được vai trò của model (đề xuất) và application code (thực thi).*
- *So sánh được Chat Completions API và Assistants API, và lựa chọn được giải pháp phù hợp cho từng tình huống cụ thể.*

8.2.1 Khởi động: Làm thế nào để "nhúng" trí thông minh vào ứng dụng?

Trong mục trước, chúng ta đã phác thảo kiến trúc tổng thể của một ứng dụng GenAI với AI Engine là một trong những thành phần cốt lõi. Nhưng chính xác thì AI Engine hoạt động như thế nào? Làm thế nào để ứng dụng của bạn – vốn được viết bằng Python, JavaScript hay bất kỳ ngôn ngữ nào – có thể "nói chuyện" được với một mô hình ngôn ngữ lớn đang chạy trên đám mây?

Tin tốt là: tích hợp AI vào ứng dụng không đòi hỏi bạn phải là chuyên gia về học sâu. Các nhà cung cấp LLM đã đóng gói sức mạnh của mô hình đằng sau những API REST quen thuộc – tương tự như cách bạn tích hợp dịch vụ thanh toán (Stripe), gửi email (SendGrid), hay lưu trữ file (Amazon S3). Bạn gửi một HTTP request chứa prompt và nhận về một HTTP response chứa câu trả lời. Điều khác biệt nằm ở chỗ: response đó không được xác định trước bởi code của bạn, mà được sinh ra bởi mô hình tại thời điểm request – với tất cả sức mạnh và cả sự không chắc chắn đi kèm.

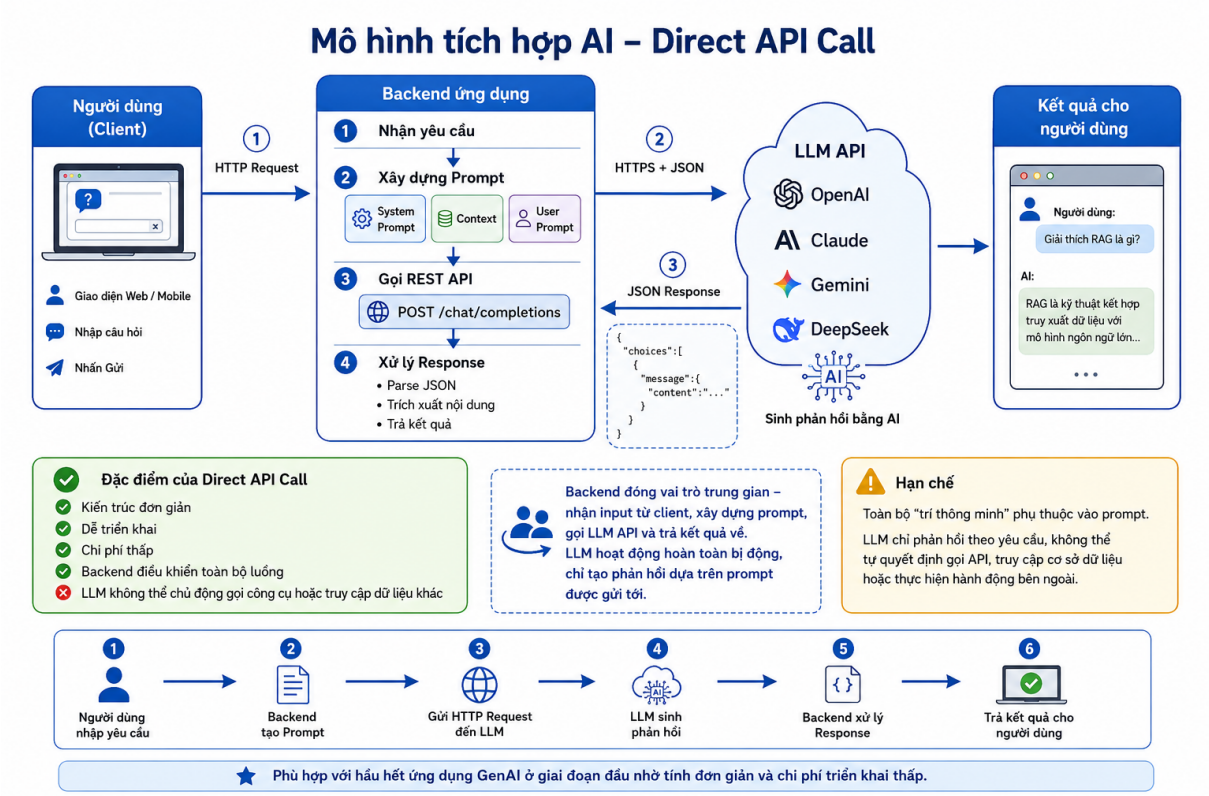
Mục này trình bày ba mô hình tích hợp AI chính, từ đơn giản đến phức tạp, cùng với các nguyên tắc thiết kế API và xử lý lỗi cho ứng dụng GenAI.

8.2.2 Mô hình 1: Gọi API trực tiếp (Direct API Call)

Đây là mô hình tích hợp đơn giản nhất và cũng là phổ biến nhất. Luồng hoạt động như sau: (1) Ứng dụng nhận input từ người dùng; (2) Backend xây dựng prompt (có thể bao gồm system message, context, và user message); (3) Backend gửi HTTP request đến LLM API (ví dụ: OpenAI Chat Completions API); (4) Backend nhận response, trích xuất nội dung, và trả về cho người dùng.

Kiến trúc này phù hợp với phần lớn ứng dụng GenAI ở giai đoạn đầu vì tính đơn giản và chi phí triển khai thấp. Tuy nhiên, nó có một hạn chế quan trọng: toàn bộ "trí thông minh" nằm trong prompt – ứng dụng không có cơ chế để mô hình chủ động yêu cầu thêm dữ liệu hoặc gọi đến các chức năng khác của hệ thống.

Hình 8.3 minh họa luồng tương tác trong mô hình gọi API trực tiếp.



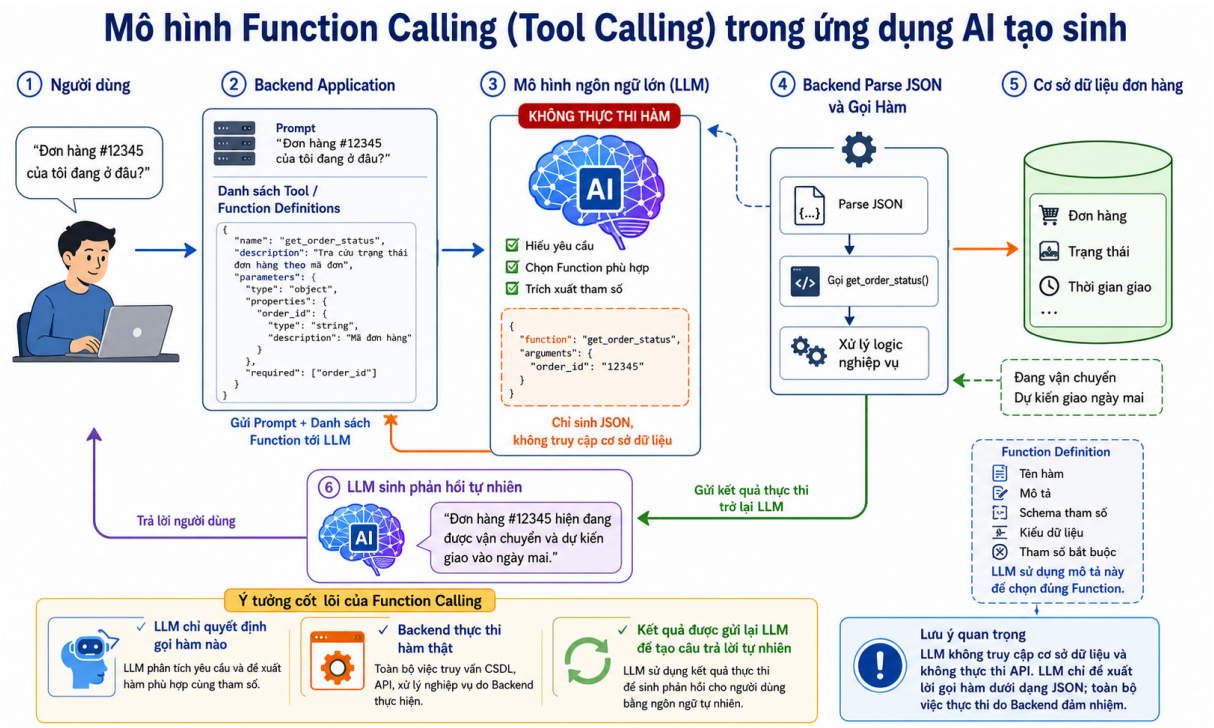
Hình 8.3: Mô hình tích hợp Direct API Call: Backend đóng vai trò trung gian – nhận input từ client, xây dựng prompt, gọi LLM API, và trả kết quả về. LLM hoàn toàn bị động, chỉ phản hồi dựa trên prompt được cung cấp.

8.2.3 Mô hình 2: Function Calling

Function calling (còn được gọi là tool calling) là một bước tiến quan trọng trong tích hợp AI. Thay vì chỉ sinh văn bản, mô hình có thể "đề xuất" gọi một hàm cụ thể với các tham số phù hợp – và application code sẽ là bên thực thi hàm đó. Điều này cho phép ứng dụng GenAI tương tác với cơ sở dữ liệu, gọi API bên ngoài, hoặc thực hiện các thao tác mà bản thân mô hình không thể làm được.

Cần nhấn mạnh một điểm quan trọng: **mô hình không thực thi hàm**. Nó chỉ phân tích input của người dùng, xác định hàm nào phù hợp (nếu có), và sinh ra một JSON object chứa tên hàm và các tham số. Application code parse JSON đó, thực thi hàm tương ứng, và – nếu cần – gửi kết quả trở lại cho mô hình để tiếp tục sinh phản hồi.

Để hiểu rõ hơn, hãy xem xét một ví dụ cụ thể. Giả sử bạn đang xây dựng ứng dụng chăm sóc khách hàng có tích hợp AI. Khi người dùng hỏi: "Đơn hàng #12345 của tôi đang ở đâu?", mô hình cần truy vấn cơ sở dữ liệu đơn hàng. Với function calling, luồng xử lý sẽ như được minh họa trong Hình 8.4.



Hình 8.4: Luồng xử lý Function Calling: (1) User gửi câu hỏi; (2) Backend gửi prompt kèm danh sách function definitions cho LLM; (3) LLM phân tích và trả về JSON với function name và parameters thay vì văn bản thông thường; (4) Backend parse JSON và thực thi hàm tương ứng (ví dụ: truy vấn database); (5) Backend gửi kết quả thực thi trở lại LLM để sinh câu trả lời cuối cùng bằng ngôn ngữ tự nhiên.

Đoạn mã 8.1 minh họa cách định nghĩa một function cho OpenAI API và xử lý phản hồi function calling trong Python. Đây là đoạn mã back-end xử lý yêu cầu tra cứu đơn hàng – một ví dụ điển hình cho thấy function calling cho phép mô hình LLM chuyển từ "nói về dữ liệu" sang "tương tác với dữ liệu thật."

Đoạn mã trên thể hiện ba điểm quan trọng của cơ chế function calling. **Thứ nhất**, việc định nghĩa hàm (`functions`) tuân theo một schema chặt chẽ với `name`, `description`, và `parameters` – mô tả càng rõ ràng, model càng có khả năng chọn đúng hàm và điền đúng tham số. **Thứ hai**, tham số `function\_call="auto"` cho phép model tự quyết

Mã nguồn 8.1: Định nghĩa function và xử lý function calling với OpenAI API

```python
import openai
import json

# Định nghĩa function mà model có thể "đề xuất" gọi
functions = [
    {
        "name": "get_order_status",
        "description": "Tra cứu trạng thái đơn hàng theo mã đơn",
        "parameters": {
            "type": "object",
            "properties": {
                "order_id": {
                    "type": "string",
                    "description": "Mã đơn hàng, ví dụ: ORD-12345"
                }
            },
            "required": ["order_id"]
        }
    }
]

def query_order_status(user_message: str) -> str:
    """Gọi LLM để xác định xem có cần tra cứu đơn hàng không."""
    response = openai.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": user_message}],
        functions=functions,
        function_call="auto"  # Model tự quyết định có gọi function hay không
    )

    choice = response.choices[0]

    # Nếu model quyết định cần gọi function
    if choice.finish_reason == "function_call":
        func_name = choice.message.function_call.name
        args = json.loads(choice.message.function_call.arguments)

        if func_name == "get_order_status":
            # Thực thi hàm thật -- truy vấn database
            result = lookup_order_in_database(args["order_id"])
            return f"Đơn hàng {args['order_id']}: {result}"

    # Nếu model không cần gọi function, trả về text response
    return choice.message.content
```

định: nếu câu hỏi không liên quan đến tra cứu đơn hàng (ví dụ "Chính sách đổi trả là gì?"), model sẽ trả lời trực tiếp mà không gọi hàm. **Thứ ba**, việc thực thi hàm thật (`lookup\_order\_in\_database`) diễn ra trong application code, không phải trong model – model chỉ chịu trách nhiệm xác định *rằng* cần tra cứu đơn hàng và *mã đơn nào* cần tra cứu.

Lưu ý: Đoạn mã trên sử dụng cú pháp `functions` của OpenAI Python SDK phiên bản cũ (< 1.0). Từ phiên bản 1.0 trở đi, OpenAI đã thống nhất function calling vào API `tools` với cấu trúc tương tự nhưng sử dụng tham số `tools` thay vì `functions`, và `tool\_choice` thay vì `function\_call`. Sinh viên nên kiểm tra phiên bản SDK đang sử dụng và tham khảo tài liệu chính thức của OpenAI để biết cú pháp mới nhất.

8.2.4 Mô hình 3: Tích hợp dựa trên Agent (Agent-based Integration)

Khi ứng dụng trở nên phức tạp hơn – với nhiều công cụ, nhiều bước xử lý và nhu cầu ra quyết định động – mô hình function calling đơn lẻ có thể không đủ. Đây là lúc tích hợp dựa trên agent (agent-based integration) phát huy giá trị.

AI agent là một hệ thống có khả năng: (1) phân tích input người dùng và xác định mục tiêu; (2) lập kế hoạch – chia mục tiêu thành các bước con; (3) lựa chọn và sử dụng công cụ phù hợp cho từng bước; (4) đánh giá kết quả trung gian và điều chỉnh kế hoạch nếu cần; (5) tổng hợp kết quả cuối cùng thành phản hồi có ý nghĩa. Thay vì một lần gọi LLM $\rightarrow$ một lần thực thi hàm $\rightarrow$ kết thúc, agent có thể thực hiện nhiều vòng lặp suy nghĩ – hành động – quan sát cho đến khi đạt được mục tiêu.

Trong kiến trúc tích hợp agent, API Gateway vẫn đóng vai trò trung tâm, nhưng backend services được tổ chức thành một hệ thống gồm: Agent Controller (điều phối luồng suy nghĩ – hành động), Tool Registry (danh sách các công cụ agent có thể sử dụng), Memory Manager (quản lý ngữ cảnh ngắn hạn và dài hạn), và Execution Engine (thực thi các hành động được agent đề xuất).

Bảng 8.2 so sánh ba mô hình tích hợp trên các tiêu chí quan trọng.

Tiêu chí	**Direct API Call**	**Function Calling**	**Agent-based**
Độ phức tạp triển khai	Thấp – gửi prompt, nhận text	Trung bình – cần định nghĩa functions và xử lý JSON	Cao – cần orchestration, memory, tool registry
Khả năng tương tác với hệ thống	Không – model chỉ sinh văn bản	Có – model đề xuất gọi hàm, app thực thi	Mạnh – agent tự chọn và phối hợp nhiều công cụ
Tính tự chủ	Không – hoàn toàn bị động	Thấp – chỉ đề xuất, không tự thực thi	Cao – tự lập kế hoạch và điều chỉnh
Chi phí vận hành	Thấp – một lần gọi API mỗi lượt	Trung bình – có thể cần 2-3 lần gọi	Cao – nhiều vòng lặp, nhiều lần gọi API
Khả năng kiểm soát	Cao – output được kiểm soát chặt qua prompt	Cao – app code quyết định có thực thi hàm không	Trung bình – agent có thể chọn hướng đi không mong đợi
Tình huống phù hợp	Hỏi đáp đơn giản, sinh nội dung, tóm tắt	Tra cứu dữ liệu, thao tác CRUD, tích hợp API	Tác vụ phức tạp nhiều bước, tự động hóa quy trình, trợ lý thông minh

Bảng 8.2: So sánh ba mô hình tích hợp AI: Direct API Call, Function Calling, Agent-based

Một nguyên tắc quan trọng khi lựa chọn mô hình tích hợp: **luôn bắt đầu với mô**

hình đơn giản nhất có thể giải quyết được bài toán. Đừng nhảy thẳng vào agent-based integration nếu một lần gọi API trực tiếp với prompt tốt đã cho kết quả chấp nhận được. Sự phức tạp tăng thêm chỉ xứng đáng khi nó mang lại giá trị tương xứng – về chất lượng đầu ra, khả năng tự động hóa, hoặc trải nghiệm người dùng.

8.2.5 Thiết kế API cho ứng dụng GenAI

API là "hợp đồng" giữa frontend và backend, giữa các service với nhau, và giữa hệ thống của bạn với thế giới bên ngoài. Với ứng dụng GenAI, thiết kế API cần giải quyết thêm một số thách thức đặc thù.

a. Cấu trúc request và response

Một API endpoint cho GenAI thường có cấu trúc request gồm ba thành phần chính: (1) `system\_context` – ngữ cảnh và quy tắc áp dụng cho toàn bộ phiên; (2) `messages` – lịch sử hội thoại giữa user và assistant; (3) `params` – các tham số điều khiển hành vi mô hình như `temperature`, `max\_tokens`, và các tùy chọn đặc thù của ứng dụng.

Response cũng cần được thiết kế cẩn thận hơn so với API truyền thống. Ngoài nội dung văn bản (`content`), response nên bao gồm: `finish\_reason` – lý do mô hình dừng sinh (hoàn thành, đạt giới hạn token, bị chặn bởi content filter); `usage` – thống kê token sử dụng (quan trọng cho việc theo dõi chi phí); và `metadata` – thông tin bổ sung như model version, timestamp, và request ID để phục vụ debugging.

b. Xử lý lỗi và retry logic

Gọi LLM API khác với gọi một API truyền thống ở chỗ: lỗi không chỉ đến từ hạ tầng (network timeout, server error) mà còn từ chính nội dung (content filter trigger, token limit exceeded). Một chiến lược xử lý lỗi cho GenAI API cần bao quát các tình huống sau:

- **Rate Limit (HTTP 429)**: bạn đã vượt quá giới hạn request trên một đơn vị thời gian. Chiến lược: exponential backoff với jitter – đợi 1 giây, nếu vẫn lỗi đợi 2 giây, 4 giây, 8 giây... cộng thêm một khoảng ngẫu nhiên nhỏ để tránh các request retry đồng loạt.
- **Server Error (HTTP 5xx)**: lỗi từ phía nhà cung cấp. Chiến lược: retry với exponential backoff, nhưng giới hạn số lần retry (thường là 3) và có cơ chế circuit breaker để ngừng gọi nếu tỉ lệ lỗi vượt ngưỡng.
- **Content Filter (finish_reason = "content_filter")**: input hoặc output vi phạm chính sách nội dung. Chiến lược: không retry với cùng input; thông báo cho người dùng rằng nội dung không phù hợp và gợi ý cách diễn đạt lại.
- **Token Limit (finish_reason = "length")**: output bị cắt vì đạt giới hạn token. Chiến lược: nếu phù hợp, tự động gửi request tiếp theo với prompt "tiếp tục từ..."; hoặc thông báo cho người dùng và đề xuất thu hẹp câu hỏi.
- **Invalid JSON (khi dùng function calling)**: model trả về JSON không hợp lệ. Chiến lược: retry với prompt bổ sung yêu cầu model sửa lỗi JSON; nếu vẫn thất bại sau 2 lần, fallback về text response.

c. Bảo vệ API endpoint

Khi bạn xây dựng một API endpoint đứng giữa người dùng và LLM provider, endpoint đó trở thành mục tiêu tấn công. Ngoài các biện pháp bảo mật API tiêu chuẩn (TLS, authentication, rate limiting), ứng dụng GenAI cần đặc biệt chú ý đến:

- **Không để lộ API key**: LLM API key phải được lưu trữ trong environment variables hoặc secrets manager của server, không bao giờ xuất hiện trong client-side code hoặc public repository.
- **Giới hạn độ dài input**: đặt giới hạn cứng cho độ dài user input để ngăn chặn tấn công từ chối dịch vụ (DoS) thông qua việc gửi prompt cực dài làm tiêu tốn token và chi phí.
- **Kiểm soát chi phí**: thiết lập ngân sách token hàng ngày cho mỗi người dùng hoặc mỗi API key. Nếu một người dùng tiêu thụ token bất thường, hệ thống sẽ tự động giới hạn hoặc cảnh báo.
- **Lọc input/output**: luôn chạy input qua moderation API trước khi gửi đến LLM, và kiểm tra output trước khi trả về cho người dùng.

8.2.6 Chat Completions API và Assistants API

OpenAI cung cấp hai API chính cho tích hợp LLM, mỗi loại phù hợp với các tình huống khác nhau. Việc hiểu rõ sự khác biệt giúp bạn chọn đúng công cụ cho đúng việc.

Chat Completions API (`/v1/chat/completions`) là API cơ bản nhất: bạn gửi một danh sách messages (với các role `system`, `user`, `assistant`) và nhận về một message response. API này phù hợp cho các tương tác độc lập, không yêu cầu duy trì trạng thái phức tạp giữa các lần gọi. Toàn bộ ngữ cảnh cần được gửi lại trong mỗi request – điều này có nghĩa là bạn (với tư cách là developer) chịu trách nhiệm quản lý lịch sử hội thoại và quyết định thông tin nào cần giữ lại khi context window sắp đầy.

Assistants API (`/v1/assistants`) cung cấp một lớp trừu tượng cao hơn: bạn tạo một assistant với instructions, model, và tools (functions, code interpreter, file search) được cấu hình sẵn; OpenAI tự động quản lý conversation threads và context memory. API này phù hợp cho các ứng dụng cần duy trì hội thoại dài, truy cập file, hoặc thực thi code trong môi trường sandbox. Tuy nhiên, sự tiện lợi này đi kèm với ít quyền kiểm soát hơn đối với cách context được quản lý và chi phí có thể cao hơn do cơ chế lưu trữ thread phía server.

Bảng 8.3 tóm tắt sự khác biệt chính giữa hai API.

8.2.7 Từ tích hợp đến kiểm thử

Ba mô hình tích hợp – Direct API Call, Function Calling, và Agent-based – cung cấp một phổ lựa chọn từ đơn giản đến phức tạp. Lựa chọn đúng mô hình cho đúng bài toán là một kỹ năng quan trọng, và kỹ năng này chỉ có được thông qua thực hành. Tuy nhiên, dù chọn mô hình nào, bạn cũng sẽ đối mặt với cùng một câu hỏi: làm thế nào để biết ứng dụng của bạn đang hoạt động tốt? Đây là lúc kiểm thử và đánh giá – chủ đề của mục tiếp theo – trở nên thiết yếu.

Trước khi chuyển sang mục tiếp theo, hãy tự kiểm tra: Trong mô hình function calling, tại sao việc model *không* thực thi hàm mà chỉ đề xuất gọi hàm lại là một quyết định thiết kế quan trọng? Điều gì sẽ xảy ra nếu model được phép tự động thực thi bất kỳ

Tiêu chí	Chat Completions API	Assistants API
Quản lý context	Developer tự quản lý – gửi toàn bộ messages mỗi request	OpenAI quản lý – context tự động duy trì qua threads
Công cụ tích hợp	Function calling (developer định nghĩa)	Code Interpreter, File Search, Functions (tích hợp sẵn)
Truy cập file	Không hỗ trợ trực tiếp	Hỗ trợ – upload file, model tự tìm kiếm và trích xuất
Độ phức tạp code	Đơn giản – gửi request, nhận response	Phức tạp hơn – cần quản lý assistant, thread, run lifecycle
Chi phí	Token input + output	Token input + output + phí lưu trữ file (nếu dùng)
Kiểm soát	Cao – toàn quyền quyết định context	Trung bình – context do OpenAI quản lý
Tình huống phù hợp	Chatbot đơn giản, sinh nội dung, Q&A một lượt	Trợ lý cá nhân, phân tích tài liệu, hội thoại dài nhiều phiên

Bảng 8.3: So sánh Chat Completions API và Assistants API

hàm nào nó muốn?

8.3 Kiểm thử và đánh giá ứng dụng AI

Sau khi hoàn thành mục này, sinh viên có thể:

- *Phân biệt được hai phạm trù kiểm thử cho ứng dụng GenAI: kiểm thử phần mềm truyền thống và kiểm thử đặc thù AI.*
- *Giải thích được các thách thức chính khi kiểm thử hệ thống có đầu ra không xác định (non-deterministic).*
- *Mô tả được các chỉ số đánh giá chất lượng mô hình: ROUGE, BLEU, và các phương pháp đánh giá dựa trên con người.*
- *Nhận diện được hiện tượng hallucination, nguyên nhân và các chiến lược giảm thiểu.*
- *Áp dụng được quy trình đánh giá ứng dụng GenAI: xác định chỉ số → tạo tập dữ liệu kiểm thử → chạy đánh giá → phân tích kết quả → cải thiện.*

8.3.1 Khởi động: Làm sao biết AI trả lời đúng?

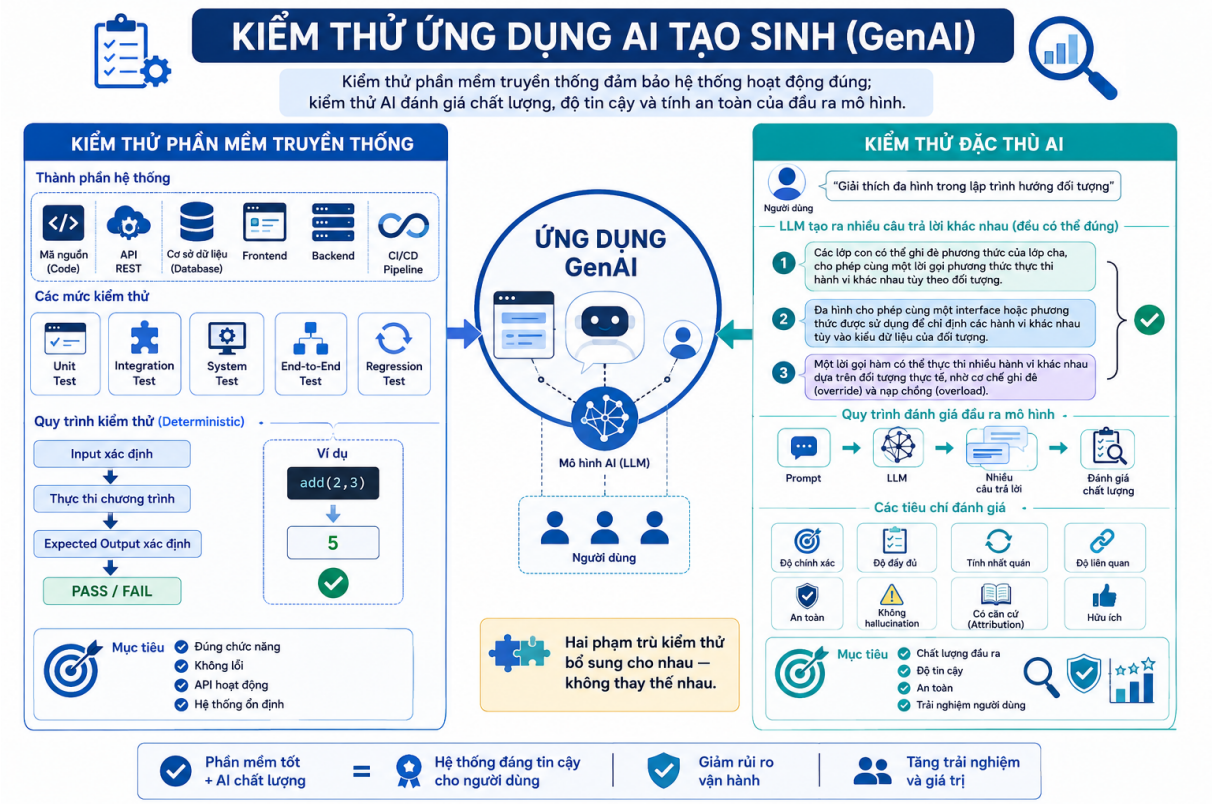
Hãy thử một thí nghiệm nhỏ. Mở một ứng dụng chat AI bất kỳ và hỏi cùng một câu hỏi ba lần. Bạn có nhận được ba câu trả lời giống hệt nhau không? Rất có thể là không. Mỗi lần, mô hình có thể chọn từ ngữ khác nhau, nhấn mạnh khía cạnh khác nhau, hoặc thậm chí – trong một số trường hợp – đưa ra thông tin mâu thuẫn nhau. Đây chính là thách thức cốt lõi của kiểm thử ứng dụng AI: **làm thế nào để kiểm thử một hệ thống mà đầu ra của nó không được xác định trước?**

Trong kiểm thử phần mềm truyền thống, bạn viết một test case với input xác định và expected output xác định. Hàm `add(2,3)` phải trả về `5` – không hơn, không kém. Nhưng với LLM, câu hỏi "Giải thích khái niệm đa hình trong lập trình hướng đối tượng" có thể nhận được hàng trăm câu trả lời khác nhau, tất cả đều có thể được coi là "đúng". Vậy tiêu chí nào để đánh giá?

Mục này trình bày một khung kiểm thử và đánh giá toàn diện cho ứng dụng GenAI, bao gồm cả các phương pháp kiểm thử truyền thống được điều chỉnh cho bối cảnh AI, lẫn các kỹ thuật đánh giá đặc thù cho mô hình ngôn ngữ.

8.3.2 Hai phạm trù kiểm thử cho ứng dụng GenAI

Khi kiểm thử một ứng dụng GenAI, bạn cần giải quyết đồng thời hai bài toán khác nhau về bản chất. Hình 8.5 minh họa sự phân tách này.



Hình 8.5: Hai phạm trù kiểm thử cho ứng dụng GenAI: (trái) Kiểm thử phần mềm truyền thống – tập trung vào code, API, và tích hợp hệ thống; (phải) Kiểm thử đặc thù AI – tập trung vào chất lượng, độ tin cậy và an toàn của đầu ra mô hình. Hai phạm trù bổ sung cho nhau, không thay thế nhau.

a. Phạm trù 1: Kiểm thử phần mềm truyền thống

Đây là những gì bạn vẫn làm với mọi dự án phần mềm – và chúng vẫn hoàn toàn cần thiết cho ứng dụng GenAI. Các mức kiểm thử bao gồm:

Unit Tests: kiểm tra từng hàm, từng module riêng lẻ. Trong ứng dụng GenAI, unit tests áp dụng cho các thành phần không liên quan đến LLM như: hàm xử lý input, hàm parse JSON response, hàm quản lý session, và các utility functions. Ví dụ: bạn có thể viết unit test để xác nhận rằng hàm `parse\_function\_call\_response()` xử lý đúng cả JSON hợp lệ lẫn JSON bị lỗi cú pháp.

Integration Tests: kiểm tra sự tương tác giữa các module. Với ứng dụng GenAI, integration tests nên bao gồm: kiểm tra luồng dữ liệu từ API Gateway đến Backend Services, kiểm tra kết nối đến LLM API (với test prompts đơn giản, không tốn nhiều token), và kiểm tra tương tác giữa các service trong pipeline RAG.

End-to-End Tests: kiểm tra toàn bộ luồng người dùng. Trong bối cảnh GenAI,

E2E tests có thể bao gồm: gửi một câu hỏi từ client, xác nhận response được trả về trong thời gian chấp nhận được, và kiểm tra rằng response không rỗng và có định dạng đúng. Tuy nhiên, E2E tests cho GenAI không nên kiểm tra *nội dung* của response – vì điều đó thuộc về phạm trù thứ hai.

b. Phạm trù 2: Kiểm thử đặc thù AI

Đây là những bài toán kiểm thử mới, phát sinh từ bản chất xác suất và không xác định của mô hình ngôn ngữ. Các khía cạnh chính bao gồm:

- **Độ chính xác (Accuracy)**: câu trả lời có đúng về mặt thực tế không? Có dựa trên tài liệu nguồn (grounded) không?
- **Độ phù hợp (Relevance)**: câu trả lời có liên quan đến câu hỏi không? Có đi lạc đề không?
- **Tính nhất quán (Consistency)**: với các câu hỏi tương tự nhau, câu trả lời có nhất quán không?
- **An toàn (Safety)**: câu trả lời có chứa nội dung độc hại, phân biệt hoặc không phù hợp không?
- **Hallucination**: mô hình có bịa đặt thông tin không có trong dữ liệu nguồn không?
- **Thiên kiến (Bias)**: câu trả lời có thể hiện thiên kiến về giới tính, chủng tộc, hoặc các yếu tố nhân khẩu học khác không?

8.3.3 Thách thức của kiểm thử đầu ra không xác định

Tính không xác định (non-determinism) của LLM là thách thức lớn nhất trong kiểm thử ứng dụng GenAI. Nó xuất phát từ hai nguồn chính. **Thứ nhất**, bản thân quá trình sinh văn bản của LLM là xác suất: ở mỗi bước, mô hình chọn token tiếp theo dựa trên phân phối xác suất, và cùng một input có thể dẫn đến các output khác nhau qua các lần chạy. **Thứ hai**, ngay cả khi bạn đặt `temperature=0` để giảm tính ngẫu nhiên, các nhà cung cấp LLM có thể cập nhật phiên bản mô hình phía sau API mà không thông báo – dẫn đến việc cùng một prompt cho ra kết quả khác nhau giữa hai ngày.

Để đối phó với thách thức này, có ba chiến lược chính:

1. **Đánh giá dựa trên tiêu chí (criterion-based evaluation)**: thay vì so sánh với một "đáp án đúng" duy nhất, bạn định nghĩa một tập tiêu chí (criteria) và đánh giá xem response có đáp ứng từng tiêu chí hay không. Ví dụ: với câu hỏi "Giải thích khái niệm kế thừa", tiêu chí có thể là: (a) nêu được định nghĩa; (b) cho ít nhất một ví dụ cụ thể; (c) đề cập đến lợi ích của kế thừa; (d) không chứa thông tin sai.
2. **Đánh giá bởi LLM (LLM-as-judge)**: sử dụng một mô hình LLM khác (thường là mô hình mạnh hơn) để đánh giá chất lượng đầu ra của mô hình đang kiểm thử. Phương pháp này có thể mở rộng tốt nhưng cần được kiểm chứng vì bản thân "giám khảo" cũng có thể mắc lỗi.
3. **Đánh giá bởi con người (human evaluation)**: đây vẫn là tiêu chuẩn vàng, đặc biệt cho các tiêu chí định tính như tính tự nhiên, giọng điệu, và mức độ hữu ích. Tuy nhiên, phương pháp này tốn kém và khó mở rộng, vì vậy thường được sử dụng cho các cột mốc đánh giá quan trọng thay vì kiểm thử hàng ngày.

8.3.4 Các chỉ số đánh giá chất lượng mô hình

Để có thể đo lường một cách khách quan, cộng đồng nghiên cứu đã phát triển nhiều chỉ số (metrics) khác nhau. Mỗi chỉ số được thiết kế cho một loại tác vụ cụ thể và có những giới hạn riêng. Bảng 8.4 tóm tắt các chỉ số phổ biến nhất.

Chỉ số	Tên đầy đủ	Áp dụng cho	Cách hoạt động
ROUGE	Recall-Oriented Understudy for Gisting Evaluation	Tóm tắt văn bản	So sánh độ chồng lấp n-gram giữa văn bản sinh ra và văn bản tham chiếu
BLEU	Bilingual Evaluation Understudy	Dịch máy	So sánh độ chồng lấp n-gram, áp dụng penalty nếu bản dịch quá ngắn
BERTScore	BERT-based Similarity Score	Đa dạng tác vụ sinh văn bản	Dùng embedding BERT để tính độ tương đồng ngữ nghĩa thay vì so khớp từ vựng
Perplexity	– (không viết tắt)	Đánh giá mô hình ngôn ngữ	Đo mức độ "ngạc nhiên" của mô hình với văn bản – càng thấp càng tốt
HELM	Holistic Evaluation of Language Models	Đánh giá toàn diện LLM	Benchmark đa chiều: accuracy, calibration, robustness, fairness, bias, toxicity, efficiency

Bảng 8.4: Các chỉ số đánh giá chất lượng mô hình ngôn ngữ phổ biến

Cần lưu ý rằng các chỉ số tự động như ROUGE và BLEU có những hạn chế đáng kể. Chúng đo lường sự tương đồng bề mặt (từ vựng, n-gram) chứ không phải sự tương đồng về ngữ nghĩa sâu. Một câu trả lời hoàn toàn chính xác nhưng dùng từ ngữ khác với bản tham chiếu có thể bị điểm thấp. Ngược lại, một câu trả lời sai nhưng dùng nhiều từ giống bản tham chiếu lại có thể được điểm cao. Vì vậy, trong thực tế, các chỉ số tự động nên được sử dụng như công cụ sàng lọc ban đầu, không phải phán quyết cuối cùng.

8.3.5 Hallucination: Nhận diện và giảm thiểu

Hallucination – hiện tượng mô hình sinh ra thông tin không có thật nhưng trình bày với giọng điệu tự tin – là một trong những vấn đề nghiêm trọng nhất của ứng dụng GenAI. Mô hình có thể bịa ra số liệu, trích dẫn giả, tên tác giả không tồn tại, hoặc thậm chí cả đoạn code tưởng chừng hợp lệ nhưng thực chất gọi đến những hàm không tồn tại.

Hallucination có ba nguyên nhân chính:

1. **Thiếu hụt kiến thức**: mô hình được huấn luyện trên dữ liệu đến một thời điểm nhất định và không biết gì về những sự kiện sau đó. Khi được hỏi về một chủ đề nằm ngoài dữ liệu huấn luyện, thay vì thừa nhận không biết, mô hình có xu hướng "điền vào chỗ trống" bằng suy đoán.
2. **Nội suy sai**: mô hình kết hợp các mảnh thông tin từ nhiều nguồn khác nhau theo cách không chính xác. Ví dụ: nó biết "Hà Nội là thủ đô của Việt Nam" và "Đà Nẵng là thành phố lớn thứ ba", rồi nội suy sai thành "Đà Nẵng là thủ đô kinh tế của Việt Nam".
3. **Áp lực tạo ra câu trả lời**: mô hình được huấn luyện để luôn tạo ra một câu trả lời có vẻ hợp lý, ngay cả khi nó không có đủ thông tin. Việc nói "Tôi không biết" không phải là hành vi mặc định của hầu hết LLM.

Các chiến lược giảm thiểu hallucination bao gồm:

- **Grounding với RAG**: thay vì dựa hoàn toàn vào kiến thức nội tại của mô hình, cung cấp cho nó các đoạn tài liệu tham khảo cụ thể và yêu cầu nó chỉ trả lời dựa trên những tài liệu đó. Đây là chiến lược hiệu quả nhất đối với ứng dụng doanh nghiệp, nơi độ chính xác của thông tin là tối quan trọng.
- **Contextual grounding check**: sau khi mô hình sinh câu trả lời, tự động kiểm tra xem mỗi khẳng định trong câu trả lời có thể được tìm thấy trong tài liệu nguồn hay không. Nếu không, đánh dấu là nghi ngờ hallucination.
- **Thiết kế prompt để khuyến khích sự trung thực**: thêm vào system prompt các chỉ dẫn như "Nếu bạn không chắc chắn về một thông tin, hãy nói rõ điều đó thay vì suy đoán" hoặc "Chỉ trả lời dựa trên thông tin được cung cấp. Nếu thông tin không có trong tài liệu, hãy trả lời: Tôi không tìm thấy thông tin này trong tài liệu được cung cấp."
- **Giảm temperature**: temperature thấp hơn (gần 0) làm giảm tính ngẫu nhiên và có xu hướng tạo ra output nhất quán hơn, an toàn hơn – mặc dù cũng kém sáng tạo hơn.

8.3.6 Kiểm thử hiệu năng cho ứng dụng GenAI

Ngoài chất lượng đầu ra, hiệu năng (performance) là một chiều đánh giá quan trọng không kém đối với ứng dụng GenAI. Người dùng có thể tha thứ cho một câu trả lời chưa hoàn hảo, nhưng sẽ nhanh chóng rời bỏ ứng dụng nếu phải đợi 10 giây cho mỗi câu trả lời.

Các chỉ số hiệu năng chính cần theo dõi bao gồm:

- **Time to First Token (TTFT)**: thời gian từ lúc gửi request đến khi nhận được token đầu tiên. Chỉ số này ảnh hưởng trực tiếp đến cảm nhận của người dùng về độ "nhanh nhạy" của ứng dụng. Với ứng dụng chat, TTFT nên dưới 1 giây.
- **Token Generation Rate**: số token được sinh ra mỗi giây sau khi token đầu tiên xuất hiện. Chỉ số này ảnh hưởng đến tốc độ hiển thị câu trả lời.
- **End-to-End Latency**: tổng thời gian từ lúc người dùng gửi câu hỏi đến khi nhận được câu trả lời hoàn chỉnh, bao gồm cả thời gian xử lý của backend và thời gian truyền mạng.
- **Throughput**: số lượng request hệ thống có thể xử lý đồng thời mà không suy giảm chất lượng.
- **API Error Rate**: tỉ lệ lời gọi LLM API thất bại (HTTP 4xx, 5xx, timeout). Tỉ lệ này nên được duy trì dưới 1% trong điều kiện vận hành bình thường.

Để kiểm thử hiệu năng, bạn cần mô phỏng tải thực tế – bao gồm cả các đỉnh tải (peak load) – và đo lường các chỉ số trên. Các công cụ như Locust, k6, hoặc Apache JMeter có thể được sử dụng cho mục đích này, với các kịch bản kiểm thử được thiết kế đặc biệt để mô phỏng hành vi người dùng GenAI (gửi câu hỏi, nhận streaming response, gửi câu hỏi tiếp theo).

8.3.7 Quy trình đánh giá ứng dụng GenAI

Để việc đánh giá có hệ thống và có thể lặp lại được, bạn cần một quy trình rõ ràng. Quy trình năm bước dưới đây cung cấp một khung làm việc cho việc đánh giá liên tục

chất lượng ứng dụng GenAI.

Bước 1: Xác định chỉ số đánh giá: Chọn các chỉ số phù hợp với mục tiêu của ứng dụng. Với ứng dụng hỏi đáp dựa trên tài liệu, các chỉ số quan trọng là: độ chính xác (câu trả lời có đúng không?), độ trung thực với nguồn (câu trả lời có dựa trên tài liệu không?), và tỉ lệ hallucination. Với ứng dụng sinh nội dung sáng tạo, các chỉ số như tính độc đáo, tính mạch lạc, và mức độ hài lòng của người dùng quan trọng hơn.

Bước 2: Tạo tập dữ liệu kiểm thử: Xây dựng một tập dữ liệu gồm các cặp (câu hỏi, câu trả lời mong đợi hoặc tiêu chí đánh giá). Tập dữ liệu này nên bao gồm: các câu hỏi điển hình (happy path), các câu hỏi khó (edge cases), các câu hỏi ngoài phạm vi (để kiểm tra khả năng từ chối), và các câu hỏi có tính khiêu khích (để kiểm tra an toàn). Tập dữ liệu cần được cập nhật định kỳ khi phát hiện các trường hợp mới.

Bước 3: Chạy đánh giá: Với mỗi câu hỏi trong tập kiểm thử, gửi đến ứng dụng và thu thập response. Đối với các chỉ số tự động (ROUGE, BLEU), chạy script đánh giá. Đối với LLM-as-judge, sử dụng một mô hình đánh giá riêng. Đối với human evaluation, tổ chức các phiên đánh giá định kỳ.

Bước 4: Phân tích kết quả: Không chỉ nhìn vào điểm số tổng thể – hãy phân tích theo từng loại câu hỏi, từng chủ đề, và từng nhóm người dùng. Một ứng dụng có điểm tổng thể 85% nhưng chỉ đạt 50% cho các câu hỏi về một chủ đề cụ thể là dấu hiệu của vấn đề cần khắc phục.

Bước 5: Cải thiện: Dựa trên kết quả phân tích, xác định các hành động cải thiện: điều chỉnh prompt, cập nhật tài liệu nguồn, thay đổi chiến lược chunking, hoặc chuyển sang mô hình mạnh hơn. Sau khi cải thiện, chạy lại đánh giá và so sánh kết quả.

8.3.8 AI hỗ trợ kiểm thử

Một xu hướng thú vị là sử dụng chính AI để kiểm thử ứng dụng AI. Các công cụ AI-powered testing có thể: phân tích code để tự động sinh unit tests (bao gồm boundary cases và failure scenarios); quét codebase để tìm các phương thức chưa được kiểm thử; phân tích error logs và xác định chính xác module gây lỗi; và đề xuất test scripts cho các endpoint mới. Điều này tạo ra một vòng phản hồi tích cực: AI giúp bạn xây dựng ứng dụng nhanh hơn, và cũng giúp bạn kiểm thử ứng dụng đó kỹ lưỡng hơn.

Tuy nhiên, cần nhấn mạnh rằng AI không thể thay thế hoàn toàn con người trong kiểm thử – đặc biệt là đối với các đánh giá về chất lượng, độ phù hợp và an toàn. AI nên được xem như một công cụ hỗ trợ đắc lực, giúp con người tập trung vào những khía cạnh đòi hỏi phán đoán và hiểu biết ngữ cảnh.

8.3.9 Từ kiểm thử đến dự án hoàn chỉnh

Kiểm thử và đánh giá không phải là bước cuối cùng trong quy trình phát triển – chúng là hoạt động xuyên suốt, từ lúc có dòng code đầu tiên cho đến khi ứng dụng vận hành trong môi trường production. Với nền tảng về quy trình xây dựng (mục 8.1), tích hợp mô hình (mục 8.2), và kiểm thử (mục này), bạn đã sẵn sàng cho bước cuối cùng: phát triển một dự án ứng dụng AI tạo sinh hoàn chỉnh, nơi tất cả những mảnh ghép này được ghép lại với nhau.

Trước khi chuyển sang mục tiếp theo, hãy tự kiểm tra: Tại sao không thể áp dụng trực tiếp quy trình viết test case truyền thống (input → expected output) cho kiểm thử

ứng dụng GenAI? Hãy nêu ít nhất hai cách tiếp cận thay thế và so sánh ưu nhược điểm của chúng.

8.4 Bài thực hành 8

Sau khi hoàn thành mục này, sinh viên có thể:

- *Mô tả được toàn bộ vòng đời phát triển một dự án GenAI, từ phát biểu bài toán đến triển khai production và giám sát sau triển khai.*
- *Xác định được các thành phần kiến trúc cần thiết cho một dự án GenAI cụ thể và giải thích được vai trò của từng thành phần.*
- *Thiết kế được lộ trình phát triển (development roadmap) cho một dự án GenAI với các cột mốc và tiêu chí đánh giá rõ ràng.*
- *Áp dụng được các nguyên tắc bảo mật, kiểm duyệt nội dung và giám sát vào một dự án GenAI thực tế.*
- *Kết nối được kiến thức từ các chương trước (kiến trúc, dữ liệu, RAG, prompt engineering) vào một dự án tích hợp hoàn chỉnh.*

8.4.1 Khởi động: Khi tất cả mảnh ghép hội tụ

Trong suốt bảy chương trước, chúng ta đã khám phá từng khía cạnh riêng lẻ của việc xây dựng ứng dụng AI tạo sinh: từ nền tảng về AI và prompt engineering (Chương 1-3), thiết kế UI/UX với AI (Chương 4), tích hợp AI vào SDLC (Chương 5), kiến trúc hệ thống (Chương 6), thiết kế dữ liệu và RAG (Chương 7), cho đến quy trình xây dựng, tích hợp và kiểm thử trong chương này. Mỗi chương giống như một mảnh ghép – và bây giờ là lúc ghép tất cả chúng lại thành một bức tranh hoàn chỉnh.

Mục này hướng dẫn bạn phát triển một dự án ứng dụng AI tạo sinh từ đầu đến cuối, sử dụng case study **SkillGenie** – trợ lý học tập thông minh – làm ví dụ minh họa xuyên suốt. Chúng ta sẽ đi qua từng bước: từ việc phát biểu bài toán, thiết kế giải pháp, xây dựng prototype, cho đến triển khai production và giám sát sau triển khai. Mục tiêu không phải là bạn sao chép nguyên mẫu dự án này, mà là bạn có thể áp dụng cùng một quy trình tư duy cho dự án của riêng mình.

8.4.2 Bước 1: Phát biểu bài toán và xác định phạm vi

Mọi dự án thành công đều bắt đầu từ một bài toán được phát biểu rõ ràng. Một bài toán mơ hồ như "xây dựng ứng dụng học tập AI" sẽ dẫn đến một giải pháp mơ hồ. Ngược lại, một bài toán được định nghĩa tốt sẽ định hướng mọi quyết định thiết kế sau này.

Với SkillGenie, bài toán được phát biểu như sau:

Problem Statement – SkillGenie
Người học lập trình – đặc biệt là người mới bắt đầu – thường gặp ba khó khăn chính: (1) không có ai giải thích khái niệm khó một cách kiên nhẫn và theo nhiều cách khác nhau cho đến khi họ hiểu; (2) không biết mình đang hiểu sai ở đâu vì thiếu phản hồi tức thì và cá nhân hóa; (3) thiếu lộ trình học tập phù hợp với trình độ và mục tiêu cá nhân. Các giải pháp hiện tại (video khóa học, sách, diễn đàn) hoặc thiếu tính tương tác, hoặc thiếu tính cá nhân hóa, hoặc cả hai.

Từ bài toán này, đội ngũ phát triển xác định phạm vi dự án (project scope) – những gì sẽ và sẽ không nằm trong phiên bản đầu tiên. Đây là một bước quan trọng để tránh "scope creep" – hiện tượng phạm vi dự án liên tục mở rộng trong quá trình phát triển.

- **Trong phạm vi**: hỗ trợ học lập trình Python cơ bản; tương tác qua văn bản (text); hai chế độ học (tự do đặt câu hỏi và học theo lộ trình có hướng dẫn); đánh giá câu trả lời tự luận ngắn; theo dõi tiến độ học tập cơ bản.
- **Ngoài phạm vi** (cho phiên bản đầu tiên): hỗ trợ ngôn ngữ lập trình khác ngoài Python; tương tác bằng giọng nói; chấm điểm code tự động (yêu cầu sandbox execution); tích hợp với LMS (Learning Management System); ứng dụng di động native.

8.4.3 Bước 2: Thiết kế trải nghiệm người dùng

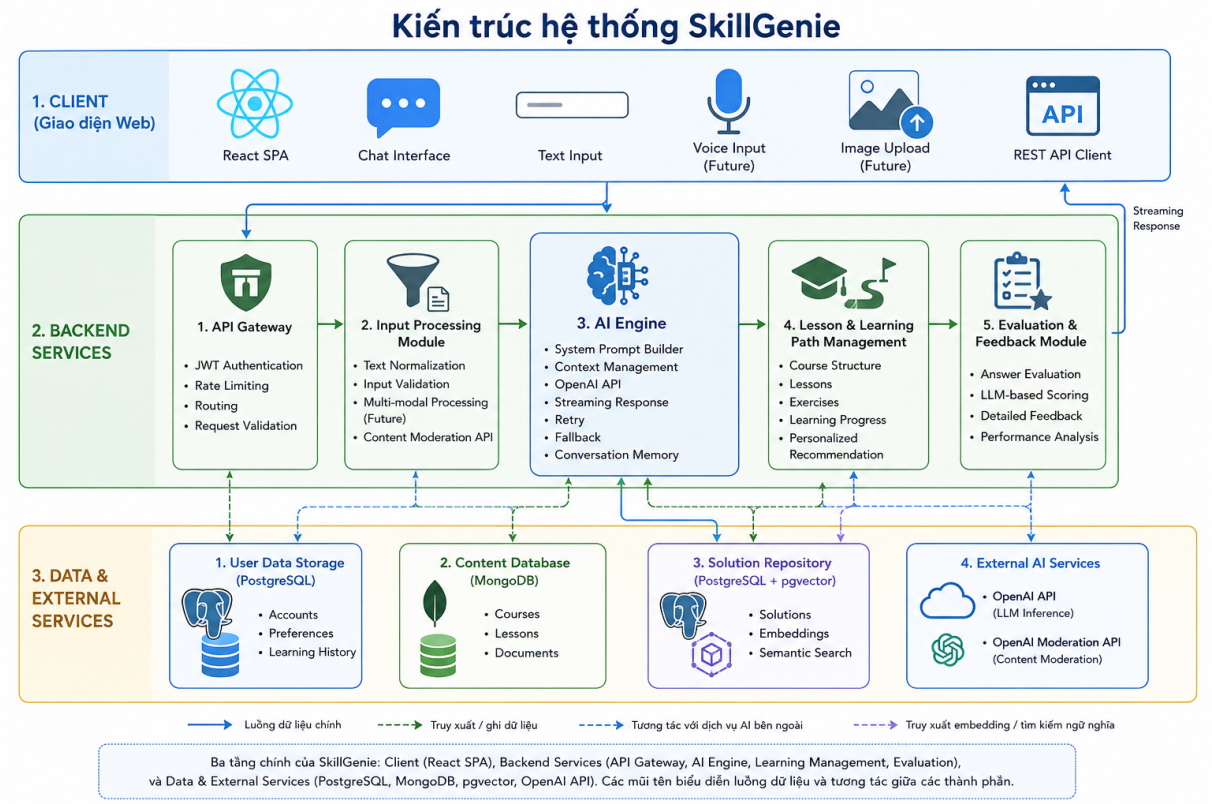
Sau khi xác định bài toán và phạm vi, bước tiếp theo là thiết kế cách người dùng sẽ tương tác với ứng dụng. Với SkillGenie, hành trình người dùng (user journey) được thiết kế thành năm giai đoạn:

1. **Chào mừng và lựa chọn**: người dùng mở ứng dụng, thấy màn hình chào mừng với các chủ đề học tập có sẵn (Python cơ bản, Cấu trúc dữ liệu, Thuật toán...), và chọn một chủ đề.
2. **Chọn chế độ học**: người dùng chọn giữa "Tự do khám phá" (đặt bất kỳ câu hỏi nào về chủ đề) hoặc "Học có hướng dẫn" (đi theo lộ trình được AI thiết kế với các bài học tuần tự).
3. **Tương tác với AI**: đây là giai đoạn cốt lõi. Người dùng đặt câu hỏi, nhận giải thích từ AI. AI không chỉ trả lời mà còn đề xuất các câu hỏi tiếp theo để khuyến khích khám phá sâu hơn. Sau mỗi chủ đề con, AI đưa ra một bài tập nhỏ để kiểm tra mức độ hiểu.
4. **Nhận đánh giá**: người dùng nộp câu trả lời cho bài tập và nhận phản hồi chi tiết từ AI – không chỉ là "đúng/sai" mà còn giải thích vì sao đúng, vì sao sai, và gợi ý cải thiện.
5. **Theo dõi tiến độ**: người dùng xem bảng điều khiển (dashboard) hiển thị các chủ đề đã hoàn thành, điểm mạnh, điểm yếu, và đề xuất chủ đề tiếp theo nên học.

Việc thiết kế user journey sớm giúp đội ngũ phát triển có một bức tranh chung về những gì cần xây dựng, đồng thời là cơ sở để ước lượng effort và lập kế hoạch phát triển.

8.4.4 Bước 3: Thiết kế kiến trúc hệ thống

Dựa trên user journey và phạm vi đã xác định, bước tiếp theo là thiết kế kiến trúc hệ thống. Kiến trúc của SkillGenie được tổ chức thành các thành phần như trong Hình 8.6.



Hình 8.6: Kiến trúc hệ thống SkillGenie: ba tầng chính – Client (giao diện web), Backend Services (xử lý logic và tích hợp AI), và Data & External Services (lưu trữ và dịch vụ bên ngoài). Các mũi tên thể hiện luồng dữ liệu chính giữa các thành phần.

Tầng Client: một ứng dụng web single-page (SPA) được xây dựng với React, giao tiếp với backend qua REST API. Giao diện tập trung vào trải nghiệm chat – tương tự như các ứng dụng nhắn tin quen thuộc – để giảm rào cản làm quen cho người dùng.

Tầng Backend Services:

- **API Gateway**: điểm vào duy nhất cho tất cả client requests, chịu trách nhiệm xác thực (JWT-based authentication), giới hạn tốc độ (rate limiting), và định tuyến (routing) đến các service phù hợp.
- **Input Processing Module**: tiếp nhận input từ người dùng, chuẩn hóa văn bản, và – trong tương lai – xử lý input đa phương thức (hình ảnh, âm thanh). Module này cũng tích hợp với moderation API để lọc nội dung không phù hợp trước khi đến LLM.
- **AI Engine**: trái tim của hệ thống, chịu trách nhiệm tương tác với LLM. Module này quản lý: xây dựng system prompt theo ngữ cảnh; gọi LLM API (OpenAI) với cơ chế retry và fallback; xử lý streaming response để hiển thị từng từ (mang lại trải nghiệm mượt mà); và quản lý context window – quyết định phần nào của lịch sử hội thoại cần giữ lại khi context sắp đầy.
- **Lesson & Learning Path Management**: quản lý nội dung học tập và lộ trình học. Module này lưu trữ cấu trúc khóa học (các chủ đề, bài học, bài tập) và sử dụng AI để đề xuất lộ trình cá nhân hóa dựa trên tiến độ và hiệu suất của từng người dùng.
- **Evaluation & Feedback Module**: chấm điểm câu trả lời của người dùng và sinh phản hồi chi tiết. Module này sử dụng LLM để đánh giá câu trả lời tự luận – một bài toán mà các hệ thống chấm điểm truyền thống (dựa trên matching chính xác) không thể giải quyết.

Tầng Data & External Services:

- **User Data Storage** (PostgreSQL): lưu trữ thông tin tài khoản, preferences, và lịch sử học tập.
- **Content Database** (MongoDB): lưu trữ nội dung khóa học, bài tập, và tài liệu tham khảo dưới dạng document linh hoạt.
- **Solution Repository** (PostgreSQL + pgvector): lưu trữ cả dữ liệu có cấu trúc (bài tập, đáp án) và vector embeddings của tài liệu học tập để hỗ trợ tìm kiếm ngữ nghĩa.
- **External AI Services**: OpenAI API cho LLM inference, OpenAI Moderation API cho lọc nội dung.

8.4.5 Bước 4: Thiết kế API

Với kiến trúc đã được xác định, bước tiếp theo là thiết kế các API endpoint – "hợp đồng" giữa frontend và backend. Bảng 8.5 liệt kê các endpoint chính của SkillGenie.

Một điểm đáng chú ý trong thiết kế API của SkillGenie là endpoint `/api/chat/send` sử dụng Server-Sent Events (SSE) để streaming response – từng token được gửi về client ngay khi được sinh ra, thay vì đợi toàn bộ response hoàn thành. Điều này cải thiện đáng kể trải nghiệm người dùng, đặc biệt với các câu trả lời dài.

Endpoint	Method	Mô tả	Request/Response chính
`/api/auth/login`	POST	Xác thực người dùng	Request: {email, password}; Response: {token, user_info}
`/api/chat/send`	POST	Gửi câu hỏi đến AI	Request: {session_id, message}; Response: streaming text/SSE
`/api/chat/history`	GET	Lấy lịch sử hội thoại	Response: {session_id, messages[]}
`/api/lessons`	GET	Lấy danh sách bài học	Response: {topics[], progress}
`/api/exercises/submit`	POST	Nộp bài tập	Request: {exercise_id, answer}; Response: {score, feedback, explanation}
`/api/progress`	GET	Lấy tiến độ học tập	Response: {completed[], strengths[], weaknesses[], recommendations[]}

Bảng 8.5: Các API endpoint chính của SkillGenie

8.4.6 Bước 5: Xây dựng prototype

Với thiết kế trong tay, đội ngũ bắt đầu xây dựng prototype. Nguyên tắc chỉ đạo là: **xây dựng phiên bản đơn giản nhất có thể kiểm chứng được giả thuyết giá trị cốt lõi**. Với SkillGenie, giả thuyết đó là: "Người học sẽ cảm thấy việc được một AI giải thích khái niệm lập trình và đưa ra phản hồi cá nhân hóa là có giá trị hơn so với việc tự đọc tài liệu."

Prototype ban đầu chỉ bao gồm:

- Một giao diện chat đơn giản (HTML + JavaScript).
- Backend Python (Flask) với một endpoint `/api/chat`.
- Tích hợp OpenAI API với một system prompt được thiết kế thủ công.
- Hỗ trợ đúng một chủ đề: Python cơ bản.
- Không có authentication, không có database (lưu session trong memory), không có streaming.

Phiên bản tối giản này được hoàn thành trong một tuần và đưa cho 10 người dùng thử nghiệm. Kết quả thử nghiệm cung cấp ba insight quan trọng: (1) người dùng đánh giá cao chất lượng giải thích – đây là tín hiệu tích cực cho giả thuyết giá trị; (2) người dùng thường không biết nên hỏi gì tiếp theo sau khi nhận được câu trả lời – đây là vấn đề cần giải quyết; (3) thời gian phản hồi 5-8 giây (không có streaming) bị coi là chậm.

Những insight này định hướng cho vòng lặp tiếp theo: thêm tính năng "câu hỏi gợi ý" và triển khai streaming response.

8.4.7 Bước 6: Tích hợp các thành phần nâng cao

Sau khi prototype được kiểm chứng, đội ngũ bắt đầu tích hợp các thành phần nâng cao hơn, dựa trên kiến thức từ các chương trước:

Tích hợp RAG (từ Chương 7): Thay vì chỉ dựa vào kiến thức nội tại của LLM, SkillGenie được trang bị một kho tri thức chứa tài liệu học tập về Python. Khi người dùng hỏi một câu, hệ thống: (1) chuyển câu hỏi thành vector embedding; (2) tìm kiếm

trong vector database (pgvector) để lấy các đoạn tài liệu liên quan nhất; (3) đưa các đoạn đó vào prompt như ngữ cảnh bổ sung; (4) yêu cầu LLM trả lời dựa trên ngữ cảnh được cung cấp. Điều này giúp giảm đáng kể hallucination và đảm bảo câu trả lời luôn bám sát nội dung khóa học.

Tích hợp RAG đánh giá (từ Chương 7): Evaluation Module sử dụng một pipeline RAG riêng cho việc chấm điểm: câu trả lời của người dùng được so sánh với đáp án mẫu và rubric chấm điểm được lưu trong vector database, sau đó LLM được cung cấp cả ba thành phần (câu trả lời, đáp án mẫu, rubric) để đưa ra đánh giá chi tiết.

Thiết kế prompt có cấu trúc (từ Chương 3): AI Engine sử dụng hệ thống prompt phân tầng: một system prompt nền tảng (định nghĩa vai trò, giọng điệu, quy tắc an toàn) kết hợp với các prompt template cho từng loại tương tác (giải thích khái niệm, chấm bài, gợi ý câu hỏi tiếp theo). Mỗi template được thiết kế với cấu trúc rõ ràng: mục tiêu, ngữ cảnh, ràng buộc, và định dạng đầu ra mong muốn.

Human-in-the-loop (từ Chương 6): Đối với các bài đánh giá quan trọng (bài kiểm tra cuối chủ đề), hệ thống được thiết kế với cơ chế HITL: AI đưa ra đánh giá sơ bộ, nhưng giáo viên có thể xem xét và điều chỉnh trước khi kết quả được gửi đến học viên. Cơ chế này vừa tận dụng được tốc độ của AI, vừa đảm bảo chất lượng và độ tin cậy của đánh giá.

8.4.8 Bước 7: Triển khai bảo mật và kiểm duyệt nội dung

Trước khi mở rộng ra nhiều người dùng hơn, đội ngũ triển khai các lớp bảo vệ sau:

1. **Input validation & moderation**: mọi input từ người dùng đều được kiểm tra qua OpenAI Moderation API trước khi đến LLM. Các input bị gắn cờ (flagged) sẽ bị chặn và người dùng nhận được thông báo yêu cầu diễn đạt lại.
2. **System prompt hardening**: system prompt bao gồm các hướng dẫn rõ ràng về phạm vi (chỉ trả lời về lập trình), giới hạn (không tư vấn pháp lý, y tế, tài chính), và cơ chế từ chối (cách từ chối lịch sự khi được hỏi ngoài phạm vi).
3. **Output filtering**: trước khi hiển thị cho người dùng, response từ LLM được kiểm tra lần cuối để phát hiện các mẫu nội dung không mong muốn (số điện thoại, địa chỉ email, hoặc nội dung có vẻ như được sinh ra từ prompt injection).
4. **Bảo vệ dữ liệu**: dữ liệu hội thoại được mã hóa khi lưu trữ; API keys được quản lý qua environment variables và secrets manager; không có dữ liệu cá nhân nào được gửi kèm trong prompt gửi đến LLM provider.

8.4.9 Bước 8: Triển khai production

Sau ba vòng lặp Build-Measure-Learn và đã có đầy đủ các lớp bảo mật, SkillGenie sẵn sàng cho production. Quy trình triển khai bao gồm:

Container hóa: Toàn bộ ứng dụng được đóng gói vào Docker containers: một container cho Flask API server, một cho PostgreSQL, một cho MongoDB, và một Nginx container làm reverse proxy. Docker Compose được sử dụng để quản lý các container trong môi trường development và staging; Kubernetes được cân nhắc cho production nếu cần scaling phức tạp hơn.

CI/CD Pipeline (GitHub Actions):

- *Pull Request*: chạy linting (flake8, black), unit tests (pytest), và integration tests

(gọi LLM API với test prompts).

- *Merge vào main*: build Docker images, push lên container registry, deploy lên staging environment.
- *Staging*: chạy smoke tests (gửi 10 câu hỏi mẫu và kiểm tra response không rỗng, không có lỗi 5xx), nếu pass thì tự động deploy lên production.
- *Production*: deploy với chiến lược rolling update để không có downtime; health check liên tục trong 5 phút đầu sau deploy; tự động rollback nếu error rate vượt ngưỡng.

Giám sát: Ngay từ ngày đầu tiên của production, các chỉ số sau được theo dõi liên tục:

- **Kỹ thuật**: CPU, memory usage, API latency (p50, p95, p99), error rate (tổng thể và riêng cho LLM API calls), token usage (tổng tokens/ngày, chi phí ước tính).
- **Sản phẩm**: số người dùng hoạt động hàng ngày (DAU), số câu hỏi trung bình mỗi phiên, tỉ lệ người dùng quay lại sau 7 ngày, điểm đánh giá trung bình của câu trả lời (từ thumbs up/down).
- **Chất lượng AI**: tỉ lệ response bị gắn cờ bởi moderation (cả input và output), phân phối điểm đánh giá từ người dùng, tỉ lệ câu trả lời bị đánh dấu là "không hữu ích", và – nếu có cơ chế – tỉ lệ hallucination được phát hiện qua contextual grounding check.

8.4.10 Bước 9: Cải thiện liên tục sau triển khai

Triển khai production không phải là đích đến cuối cùng – nó là điểm khởi đầu cho quá trình cải thiện liên tục. Dựa trên dữ liệu giám sát và phản hồi người dùng, đội ngũ SkillGenie duy trì một chu kỳ cải thiện hàng tuần:

- **Tuần 1-2**: phân tích các câu hỏi bị đánh giá thấp (thumbs down), xác định mẫu chung (có phải tất cả đều về một chủ đề? một cách hỏi?), và điều chỉnh prompt hoặc bổ sung tài liệu cho kho tri thức.
- **Tuần 3-4**: thử nghiệm A/B test – 90% người dùng dùng prompt hiện tại, 10% dùng prompt cải tiến; so sánh tỉ lệ thumbs up, thời gian tương tác trung bình, và tỉ lệ quay lại.
- **Tuần 5-6**: đánh giá chi phí – token usage có đang tăng nhanh hơn số người dùng không? Có cơ hội nào để giảm chi phí thông qua caching, prompt optimization, hoặc chuyển sang model nhỏ hơn cho các tác vụ đơn giản không?
- **Tuần 7-8**: thử nghiệm model mới – khi OpenAI phát hành model mới, chạy bộ test suite trên model mới và so sánh chất lượng/chi phí với model hiện tại trước khi quyết định chuyển đổi.

8.4.11 Bài học từ dự án thực tế

Hành trình xây dựng SkillGenie minh họa một số bài học quan trọng cho bất kỳ dự án GenAI nào:

1. **Bắt đầu nhỏ, mở rộng nhanh**. Đừng cố xây dựng mọi tính năng trong phiên bản đầu tiên. Hãy xác định giả thuyết giá trị cốt lõi, xây dựng phiên bản tối thiểu để kiểm chứng nó, và chỉ đầu tư vào các tính năng bổ sung sau khi giả thuyết đã được xác nhận.

2. **Đo lường mọi thứ**. Trong ứng dụng GenAI, bạn không thể dựa vào trực giác để đánh giá chất lượng. Bạn cần dữ liệu: tỉ lệ thumbs up/down, thời gian tương tác, tỉ lệ quay lại, phân tích lỗi. Dữ liệu sẽ cho bạn biết điều gì đang hoạt động và điều gì cần cải thiện.
3. **Thiết kế để thay đổi**. Prompt, model, và thậm chí toàn bộ provider có thể thay đổi trong tương lai. Hãy thiết kế kiến trúc sao cho việc thay đổi prompt không đòi hỏi deploy lại toàn bộ ứng dụng, và việc chuyển đổi model provider chỉ ảnh hưởng đến một module duy nhất.
4. **Bảo mật không phải là bước cuối cùng**. Đừng đợi đến trước ngày ra mắt mới nghĩ về bảo mật. Ngay từ prototype, hãy có ít nhất moderation API và system prompt cơ bản. Việc thêm bảo mật sau khi hệ thống đã phức tạp sẽ khó khăn hơn nhiều.
5. **Con người luôn trong vòng lặp**. Với các ứng dụng có tác động lớn (như đánh giá học tập), đừng để AI tự động đưa ra quyết định cuối cùng. Hãy thiết kế cơ chế để con người có thể xem xét, điều chỉnh, và – khi cần – ghi đè quyết định của AI.

8.4.12 Kết nối với dự án xuyên suốt của giáo trình

Dự án SkillGenie trong chương này là một minh họa cụ thể cho ví dụ xuyên suốt của giáo trình: **"Trợ lý phát triển phần mềm thông minh"**. Cả hai đều là ứng dụng AI tạo sinh phục vụ một nhóm người dùng chuyên biệt (lập trình viên / người học lập trình), đều yêu cầu tích hợp LLM với kho tri thức chuyên ngành, và đều đòi hỏi cơ chế đánh giá và phản hồi chất lượng cao.

Khi bạn bắt tay vào xây dựng dự án của riêng mình – cho dù đó là trợ lý phát triển phần mềm, chatbot chăm sóc khách hàng, hay công cụ phân tích tài liệu – hãy nhớ rằng quy trình và nguyên tắc là giống nhau. Điều thay đổi là domain knowledge, dữ liệu, và các quyết định thiết kế cụ thể. Chương 9 – chương cuối cùng của giáo trình – sẽ giúp bạn đưa ứng dụng đó vào vận hành thực tế với các chiến lược triển khai, giám sát và đảm bảo chất lượng trong môi trường production.

Trước khi chuyển sang phần ôn tập cuối chương, hãy tự kiểm tra: Bạn có thể kể tên chín bước trong quy trình phát triển dự án GenAI hoàn chỉnh không? Bước nào bạn cho là khó khăn nhất và tại sao?

8.4.13 Xây dựng ứng dụng hoàn chỉnh

Xây dựng hệ thống Quản lý bán hàng (tiếp tục từ Bài 7).
Xem chi tiết tại Bài thực hành 1. Nhắc lại: mọi kết quả AI phải được kiểm tra; không cung cấp dữ liệu nhạy cảm cho AI công cộng; duy trì Nhật ký sử dụng AI.

Sinh viên hoàn thiện các chức năng của hệ thống và sử dụng AI để hỗ trợ tích hợp, kiểm thử và tối ưu chương trình. AI được dùng để sinh ca kiểm thử, phát hiện lỗi, tái cấu trúc mã, sinh tài liệu API, hướng dẫn cài đặt và tài liệu kỹ thuật. Sinh viên đánh giá chất lượng, sửa lỗi và chuẩn bị sản phẩm cho giai đoạn triển khai.

a. Mục tiêu

- Hoàn thiện phạm vi chức năng đã cam kết và tích hợp các thành phần.

- Xây dựng chiến lược kiểm thử dựa trên rủi ro và yêu cầu.
- Cải thiện chất lượng, hiệu năng, bảo mật và khả năng bảo trì.
- Hoàn thiện bộ tài liệu phát hành.

b. Chuẩn bị

- Mã nguồn, backlog, kiến trúc, dữ liệu và giao diện từ các bài trước.
- Danh sách chức năng còn thiếu, lỗi tồn và nợ kỹ thuật.
- Môi trường kiểm thử tách biệt với dữ liệu thật.
- Công cụ kiểm thử, phân tích chất lượng và đo hiệu năng do nhóm lựa chọn.

c. Hướng dẫn thực hiện

Bước 1: Rà soát phạm vi và truy vết

- Đối chiếu backlog với chức năng đã hiện thực.
- Cập nhật ma trận truy vết từ yêu cầu đến mã, ca kiểm thử và kết quả.
- Phân loại nội dung bắt buộc, hoãn và loại bỏ; ghi lý do.

Bước 2: Hoàn thiện chức năng

- Hoàn thiện quản lý đơn hàng, thanh toán, tồn kho, báo cáo và các chức năng đã cam kết.
- Xử lý giao dịch, dữ liệu đồng thời, lỗi mạng/dịch vụ và quyền truy cập.
- Kiểm tra luồng liên kết giữa sản phẩm, khách hàng, đơn hàng và tồn kho.

Bước 3: Xây dựng và thực hiện kiểm thử

- Lập kế hoạch kiểm thử đơn vị, tích hợp, hệ thống, chấp nhận, bảo mật và hiệu năng theo mức rủi ro.
- Dùng AI đề xuất ca kiểm thử từ yêu cầu, sau đó loại bỏ ca trùng và bổ sung trường hợp biên.
- Tự động hóa các kiểm thử ổn định, lặp lại và có giá trị cao.
- Ghi bằng chứng, lỗi, mức nghiêm trọng và kết quả kiểm thử lại.

Bước 4: Rà soát chất lượng và bảo mật

- Kiểm tra phụ thuộc, cấu hình, bí mật, phân quyền, đầu vào và thông báo lỗi.
- Rà soát đoạn mã phức tạp, trùng lặp hoặc khó kiểm thử.
- Chỉ tái cấu trúc khi có kiểm thử bảo vệ hành vi hiện tại.

Bước 5: Đo và tối ưu

- Xác định kịch bản và số đo hiệu năng trước khi tối ưu.
- Đo thời gian đáp ứng, sử dụng tài nguyên và điểm nghẽn.
- Thực hiện thay đổi nhỏ, đo lại và ghi tác động; tránh tối ưu theo suy đoán.

Bước 6: Hoàn thiện tài liệu và bản phát hành ứng viên

- Tạo tài liệu cài đặt, cấu hình, sử dụng, API/giao diện tích hợp và xử lý sự cố.
- Lập ghi chú phát hành, giới hạn đã biết và hướng dẫn sao lưu/khôi phục.

- Đóng gói một bản phát hành ứng viên có thể cài đặt lại.

d. Prompt mẫu

Prompt 1 – Sinh ca kiểm thử:

"Dựa trên yêu cầu [nội dung], tiêu chí chấp nhận [nội dung] và giao diện hệ thống [mô tả], hãy đề xuất ca kiểm thử theo bảng: mã, mục tiêu, tiền điều kiện, dữ liệu, bước, kết quả mong đợi và mức ưu tiên. Bao gồm đường đi chính, trường hợp biên, lỗi, phân quyền và dữ liệu đồng thời. Không giả định chức năng ngoài yêu cầu."

Prompt 2 – Đề xuất tái cấu trúc:

"Hãy phân tích đoạn mã [nội dung] và các kiểm thử hiện có [nội dung]. Chỉ ra vấn đề có bằng chứng về độ phức tạp, trùng lặp, phụ thuộc hoặc khả năng kiểm thử. Đề xuất các bước tái cấu trúc nhỏ, giữ nguyên hành vi; nêu kiểm thử cần chạy sau mỗi bước."

e. Kiểm tra và hiệu chỉnh kết quả AI

- Chạy mọi ca kiểm thử và mã do AI tạo; không đánh dấu đạt nếu chỉ đọc nội dung.
- Xác minh AI không sửa hành vi nghiệp vụ trong quá trình tái cấu trúc.
- Không dùng dữ liệu sản xuất thật cho việc tạo hoặc thử nghiệm prompt.
- Kiểm tra tài liệu khớp với bản phát hành ứng viên.
- Ghi rõ lỗi nào AI phát hiện đúng, bỏ sót hoặc báo sai.

f. Sản phẩm cần nộp

- Bản phát hành ứng viên và mã nguồn có gắn phiên bản.
- Ma trận truy vết cập nhật.
- Kế hoạch, ca kiểm thử, mã kiểm thử và báo cáo kết quả.
- Báo cáo chất lượng, bảo mật và đo hiệu năng trước/sau.
- Bộ tài liệu cài đặt, sử dụng, kỹ thuật, ghi chú phát hành và Nhật ký AI của Bài 8.

g. Tiêu chí đánh giá (10 điểm)

- Mức hoàn thiện chức năng và tích hợp: 2,5 điểm.
- Chiến lược, độ bao phủ và bằng chứng kiểm thử: 2,0 điểm.
- Xử lý lỗi, bảo mật và chất lượng mã: 1,5 điểm.
- Đo lường và tối ưu có căn cứ: 1,0 điểm.
- Ma trận truy vết và quản lý bản phát hành: 1,0 điểm.
- Tài liệu sản phẩm: 1,0 điểm.
- Nhật ký AI và khả năng phản biện kết quả: 1,0 điểm.

Ôn tập cuối chương

Tóm tắt kiến thức cần nhớ

- **Quy trình năm giai đoạn xây dựng ứng dụng GenAI:** (1) Thiết kế kiến trúc ứng dụng – xác định functional/non-functional requirements và thiết kế kiến trúc client-server; (2) Xây dựng prototype hoạt động được – MVP để kiểm chứng giả thuyết giá trị; (3) Triển khai bảo mật và kiểm duyệt nội dung – input validation, moderation API, system prompt hardening, output filtering; (4) Triển khai production – Docker containerization, CI/CD pipeline, multi-region deployment; (5) Tối ưu hóa cho khả năng mở rộng và độ tin cậy – monitoring, user feedback, continuous improvement. Quy trình vận hành theo mô hình lặp Build – Measure – Learn thay vì tuyến tính.
- **Ba mô hình tích hợp AI:** (1) Direct API Call – đơn giản nhất, backend gửi prompt và nhận text, phù hợp cho Q&A và sinh nội dung đơn giản; (2) Function Calling – model đề xuất gọi hàm với tham số (sinh JSON), application code thực thi, cho phép tương tác với database và external APIs; (3) Agent-based Integration – agent tự lập kế hoạch, chọn công cụ, và thực hiện nhiều vòng lặp suy nghĩ-hành động-quan sát, phù hợp cho tác vụ phức tạp nhiều bước. Nguyên tắc: luôn bắt đầu với mô hình đơn giản nhất có thể.
- **Thiết kế API cho GenAI:** API Gateway đóng vai trò trung tâm – xác thực, rate limiting, routing. Request gồm system_context, messages (lịch sử hội thoại), và params (temperature, max_tokens). Response cần bổ sung finish_reason, usage (token count), và metadata. Chiến lược xử lý lỗi: exponential backoff với jitter cho rate limit và server error; không retry với content filter; retry có giới hạn cho invalid JSON. Chat Completions API phù hợp tương tác đơn giản; Assistants API phù hợp hội thoại dài với persistent context và tools tích hợp sẵn.
- **Hai phạm trù kiểm thử GenAI:** (1) Kiểm thử phần mềm truyền thống – unit tests cho code không liên quan LLM, integration tests cho luồng dữ liệu và kết nối, E2E tests cho toàn bộ luồng người dùng; (2) Kiểm thử đặc thù AI – accuracy, relevance, consistency, safety, hallucination, bias. Thách thức lớn nhất là tính không xác định (non-determinism) của LLM. Ba chiến lược đối phó: criterion-based evaluation, LLM-as-judge, và human evaluation (tiêu chuẩn vàng).
- **Đánh giá chất lượng mô hình:** ROUGE (tóm tắt – so sánh n-gram overlap), BLEU (dịch máy – n-gram + brevity penalty), BERTScore (ngữ nghĩa – dùng embedding thay vì từ vựng). Hạn chế: các chỉ số tự động chỉ đo tương đồng bề mặt, không nắm bắt được ngữ nghĩa sâu. HELM benchmark cung cấp đánh giá toàn diện đa chiều (accuracy, calibration, robustness, fairness, bias, toxicity, efficiency).
- **Hallucination – nhận diện và giảm thiểu:** Ba nguyên nhân chính: thiếu hụt kiến thức (ngoài training data), nội suy sai (kết hợp sai các mảnh thông tin), và áp lực tạo ra câu trả lời. Bốn chiến lược giảm thiểu: (1) Grounding với RAG – cung cấp tài liệu tham khảo và yêu cầu model chỉ trả lời dựa trên tài liệu đó; (2) Contextual grounding check – tự động kiểm tra từng khẳng định có trong nguồn không; (3) Thiết kế prompt khuyến khích trung thực – hướng dẫn model nói "không biết" khi thiếu thông tin; (4) Giảm temperature để output nhất quán và an toàn hơn.
- **Quy trình đánh giá năm bước:** (1) Xác định chỉ số đánh giá phù hợp với mục

tiêu ứng dụng; (2) Tạo tập dữ liệu kiểm thử bao gồm happy path, edge cases, out-of-scope, và adversarial questions; (3) Chạy đánh giá với các chỉ số tự động, LLM-as-judge, và/hoặc human evaluation; (4) Phân tích kết quả theo từng loại câu hỏi và chủ đề; (5) Cải thiện dựa trên phân tích và chạy lại đánh giá.

- **Quy trình chín bước phát triển dự án GenAI hoàn chỉnh:** (1) Phát biểu bài toán và xác định phạm vi; (2) Thiết kế trải nghiệm người dùng; (3) Thiết kế kiến trúc hệ thống (client, backend services, data & external services); (4) Thiết kế API; (5) Xây dựng prototype; (6) Tích hợp thành phần nâng cao (RAG, prompt có cấu trúc, HITL); (7) Triển khai bảo mật và kiểm duyệt nội dung; (8) Triển khai production (Docker, CI/CD, monitoring); (9) Cải thiện liên tục sau triển khai (phân tích phản hồi, A/B test, tối ưu chi phí, thử nghiệm model mới).

Câu hỏi ôn tập

1. Trình bày năm giai đoạn của quy trình xây dựng ứng dụng GenAI. Với mỗi giai đoạn, nêu một quyết định thiết kế quan trọng và giải thích hệ quả của quyết định đó đối với các giai đoạn sau.
2. Phân biệt yêu cầu chức năng và yêu cầu phi chức năng trong dự án GenAI. Tại sao các yêu cầu phi chức năng (như bảo mật, hiệu năng, khả năng mở rộng) lại đặc biệt quan trọng đối với ứng dụng GenAI so với ứng dụng web truyền thống? Cho ví dụ cụ thể.
3. So sánh ba mô hình tích hợp AI (Direct API Call, Function Calling, Agent-based) trên các tiêu chí: độ phức tạp triển khai, khả năng tương tác với hệ thống, chi phí vận hành, và tình huống áp dụng phù hợp. Với mỗi mô hình, đưa ra một ví dụ ứng dụng cụ thể mà mô hình đó là lựa chọn tối ưu.
4. Giải thích cơ chế function calling. Sử dụng một ví dụ cụ thể (khác với ví dụ tra cứu đơn hàng trong giáo trình) để minh họa luồng xử lý từ lúc người dùng gửi câu hỏi đến khi nhận được câu trả lời cuối cùng. Tại sao việc model không trực tiếp thực thi hàm mà chỉ đề xuất là một quyết định thiết kế quan trọng?
5. Thiết kế một chiến lược xử lý lỗi cho ứng dụng GenAI gọi OpenAI API. Chiến lược cần bao quát ít nhất năm tình huống lỗi khác nhau, với mỗi tình huống nêu rõ: (a) cách phát hiện, (b) hành động xử lý, và (c) lý do cho cách xử lý đó.
6. Phân tích ba thách thức chính của việc kiểm thử ứng dụng GenAI so với ứng dụng truyền thống. Với mỗi thách thức, đề xuất ít nhất một chiến lược hoặc công cụ để giải quyết. Đánh giá ưu điểm và hạn chế của từng chiến lược.
7. Trình bày ba chiến lược đánh giá chất lượng đầu ra LLM: criterion-based evaluation, LLM-as-judge, và human evaluation. So sánh ba chiến lược này về chi phí, khả năng mở rộng, và độ tin cậy. Trong tình huống nào bạn sẽ sử dụng kết hợp cả ba?
8. Giải thích hiện tượng hallucination trong LLM, bao gồm ba nguyên nhân chính và bốn chiến lược giảm thiểu. Với mỗi chiến lược, nêu một tình huống mà chiến lược đó đặc biệt hiệu quả và một tình huống mà nó có thể không đủ.
9. Mô tả quy trình phát triển chín bước cho dự án GenAI hoàn chỉnh. Phân tích mối quan hệ giữa bước 3 (thiết kế kiến trúc hệ thống), bước 6 (tích hợp thành phần nâng cao), và bước 8 (triển khai production) – một quyết định ở bước 3 ảnh hưởng như thế nào đến những gì có thể làm được ở bước 6 và 8?
10. Trong case study SkillGenie, đội ngũ phát triển đã phát hiện ra ba insight quan

trọng từ việc thử nghiệm prototype. Hãy phân tích từng insight: (a) tại sao insight đó quan trọng; (b) giải pháp đã được triển khai để giải quyết; (c) nếu bạn là trưởng nhóm, bạn có quyết định khác không? Tại sao?

Bài tập thực hành

1. **Xây dựng Chatbot với Function Calling:** Sử dụng Python và OpenAI API:
   (a) Thiết kế một chatbot đơn giản cho thư viện trường học. Định nghĩa ít nhất ba functions: `search\_books(query)` để tìm sách, `check\_availability(book\_id)` để kiểm tra tình trạng mượn, và `get\_due\_date(user\_id,book\_id)` để tra cứu hạn trả.
   (b) Triển khai backend với Flask hoặc FastAPI, tích hợp function calling để model có thể tự động chọn và đề xuất gọi hàm phù hợp dựa trên câu hỏi của người dùng.
   (c) Viết ít nhất 10 test prompts (bao gồm cả câu hỏi trong phạm vi và ngoài phạm vi) để kiểm tra khả năng chọn đúng hàm và điền đúng tham số.
   (d) Đánh giá kết quả: tỉ lệ chọn đúng hàm, tỉ lệ điền đúng tham số, và các trường hợp thất bại. Đề xuất cải thiện.
2. **Đánh giá và so sánh chất lượng LLM:**
   (a) Chuẩn bị một tập 20 câu hỏi về một chủ đề bạn am hiểu (ví dụ: một ngôn ngữ lập trình, một framework, hoặc một lĩnh vực chuyên môn).
   (b) Gửi từng câu hỏi đến ít nhất hai LLM khác nhau (ví dụ: GPT-4o và GPT-3.5 Turbo, hoặc GPT-4o và Claude). Với mỗi câu trả lời, đánh giá trên ba tiêu chí: (a) độ chính xác (thang 1-5); (b) độ đầy đủ (thang 1-5); (c) có hallucination không (có/không).
   (c) Tính điểm trung bình cho từng model trên từng tiêu chí. Phân tích: model nào tốt hơn? Có sự khác biệt đáng kể giữa các loại câu hỏi không?
   (d) Chọn 5 câu trả lời có hallucination và phân tích nguyên nhân: thiếu kiến thức, nội suy sai, hay áp lực tạo câu trả lời? Đề xuất cách giảm thiểu cho từng trường hợp.
3. **Thiết kế dự án GenAI:** Chọn một trong các ý tưởng sau (hoặc đề xuất ý tưởng riêng): (a) trợ lý ôn thi trắc nghiệm; (b) chatbot tư vấn sản phẩm cho cửa hàng trực tuyến; (c) công cụ phân tích và tóm tắt văn bản pháp lý.
   (a) Viết problem statement (1 đoạn) và xác định phạm vi dự án (trong phạm vi / ngoài phạm vi cho phiên bản đầu tiên).
   (b) Thiết kế user journey (3-5 bước) và phác thảo giao diện chính (có thể vẽ tay hoặc dùng công cụ wireframe).
   (c) Thiết kế kiến trúc hệ thống: vẽ sơ đồ các thành phần chính, xác định công nghệ cho từng thành phần, và giải thích luồng dữ liệu cho một tương tác điển hình.
   (d) Thiết kế ít nhất 4 API endpoints, với cấu trúc request/response cho từng endpoint.
   (e) Xác định ít nhất ba rủi ro kỹ thuật hoặc sản phẩm và đề xuất cách giảm thiểu.
4. **Kiểm thử và đánh giá một pipeline RAG:** Cho một pipeline RAG đơn giản (có thể tự xây dựng hoặc sử dụng code mẫu):

(a) Tạo một tập dữ liệu kiểm thử gồm 15 câu hỏi: 5 câu có câu trả lời trực tiếp trong tài liệu, 5 câu đòi hỏi tổng hợp từ nhiều phần, và 5 câu ngoài phạm vi tài liệu.
(b) Với mỗi câu hỏi, đánh giá: (a) retrieval quality – các chunk được truy xuất có liên quan không? (b) answer faithfulness – câu trả lời có dựa trên tài liệu được cung cấp không? (c) hallucination – có thông tin nào bịa đặt không?
(c) Tính các chỉ số: retrieval precision@k, answer accuracy, và hallucination rate. Trực quan hóa kết quả.
(d) Đề xuất ít nhất ba cải tiến cho pipeline dựa trên kết quả đánh giá. Với mỗi cải tiến, dự đoán tác động đến các chỉ số đã đo.

Câu hỏi thảo luận

1. Function calling đã thu hẹp khoảng cách giữa khả năng "nói" và khả năng "làm" của LLM. Tuy nhiên, nó cũng đặt ra câu hỏi về bảo mật: nếu model có thể đề xuất gọi bất kỳ hàm nào được định nghĩa, làm thế nào để đảm bảo nó không bị lợi dụng để gọi những hàm nguy hiểm? Thảo luận về sự cân bằng giữa khả năng (cho phép model tương tác với nhiều hệ thống) và an toàn (giới hạn những gì model có thể yêu cầu). Đề xuất một cơ chế kiểm soát cụ thể.
2. Trong bối cảnh giáo dục, việc sử dụng AI để đánh giá bài làm của sinh viên đặt ra những câu hỏi khó: AI có thể công bằng hơn con người (không bị ảnh hưởng bởi tâm trạng, định kiến) hay kém công bằng hơn (bị ảnh hưởng bởi bias trong dữ liệu huấn luyện)? Khi nào nên để AI tự động chấm điểm và khi nào cần human-in-the-loop? Thảo luận dựa trên case study SkillGenie và trải nghiệm cá nhân của bạn.
3. Các công ty công nghệ lớn đang chạy đua phát triển các AI agent có khả năng tự chủ cao – tự lập kế hoạch, tự chọn công cụ, tự thực thi. Liệu chúng ta có đang tiến tới một tương lai nơi phần lớn ứng dụng phần mềm được xây dựng bởi AI agent thay vì lập trình viên? Hay agent-based integration chỉ là một công cụ khác trong hộp công cụ của developer, giống như cách OOP hay microservices đã từng được kỳ vọng sẽ "thay đổi mọi thứ"? Lập luận của bạn cần dựa trên ít nhất ba khía cạnh: kỹ thuật, kinh tế, và đạo đức nghề nghiệp.